

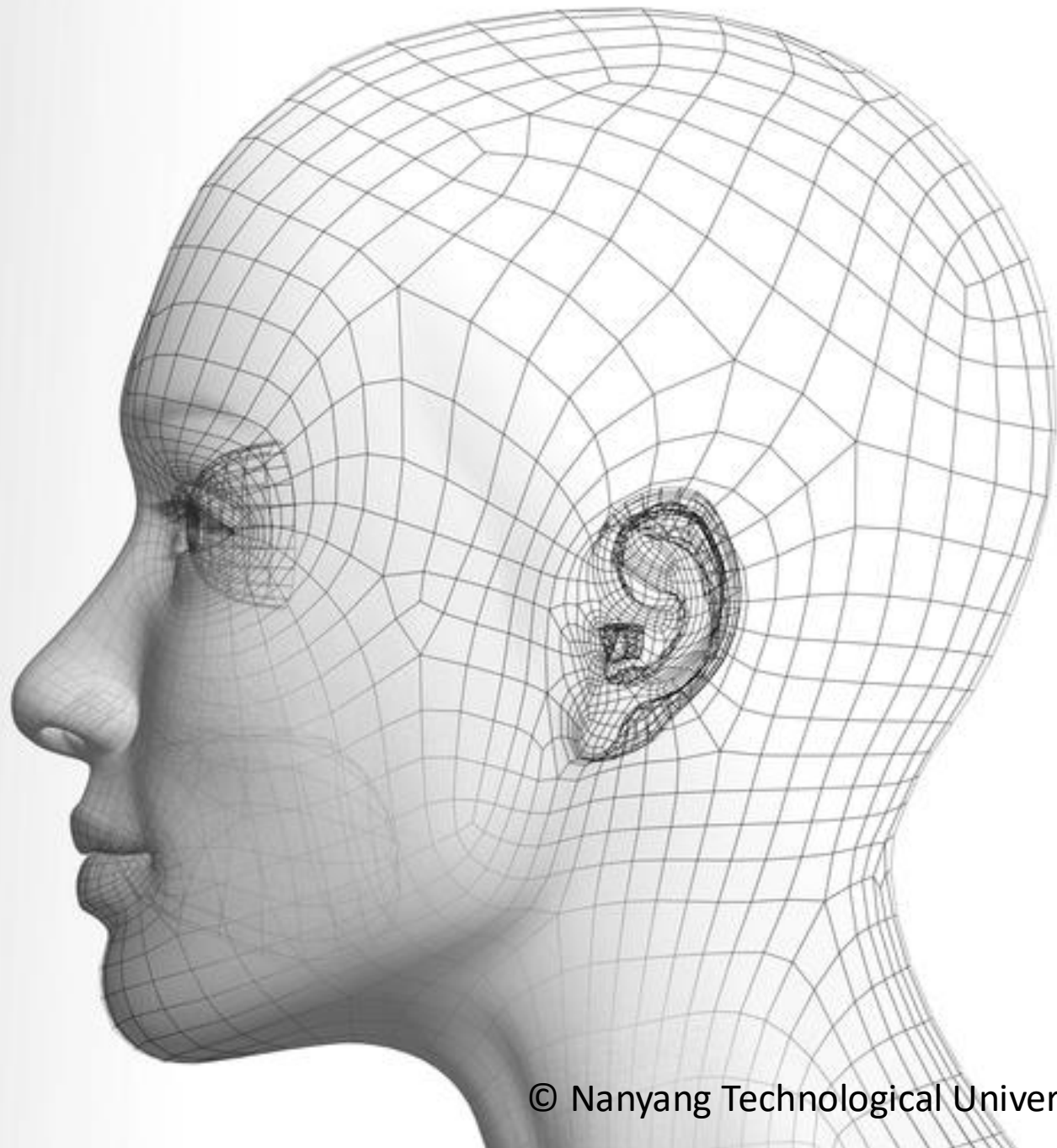
Transformers

Xingang Pan

潘新钢

<https://xingangpan.github.io/>

<https://twitter.com/XingangP>

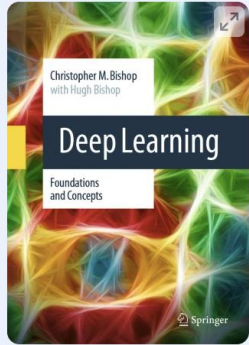


Outline

- Attention
- Transformers
- Vision Transformers

Reference book

[Home](#) > Textbook



Deep Learning

Foundations and Concepts

Textbook | © 2024

✓ Access provided by Nanyang Technological University Library

Download book PDF 

Overview

Authors: [Christopher M. Bishop](#) , [Hugh Bishop](#)

- Foundational and conceptual approach emphasizes real-world practical value of techniques for a wide range of learners
- Companion volume to the author's standard reference text Pattern Recognition and Machine Learning
- To reinforce key ideas, end-of-chapter exercises of varying difficulty are included to promote active learning

 316k Accesses  21 Citations  34 Altmetric

Go to Library Collection

<https://libguides.ntu.edu.sg/az.php?q=SpringerLink>

Search for Chris Bishop

Download the e-book for free

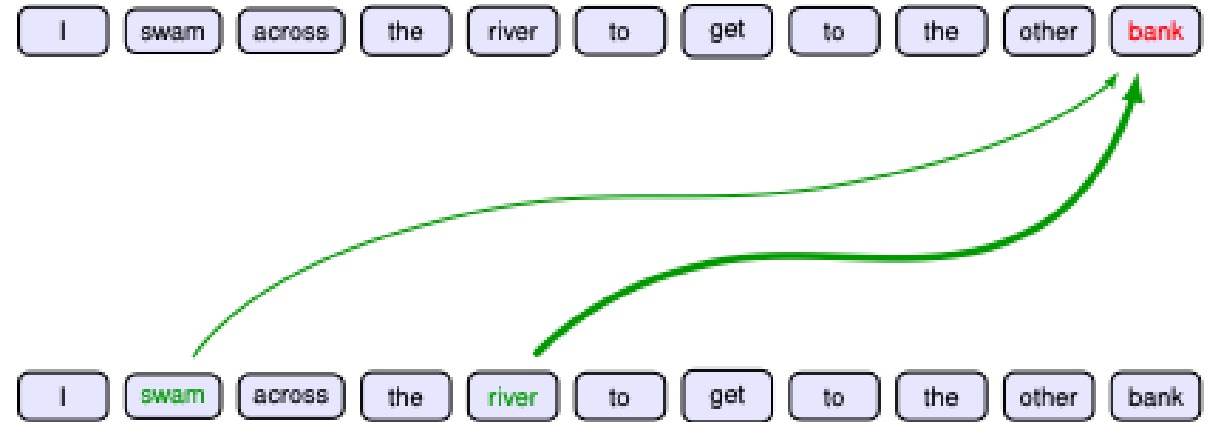
Attention



"Where's Waldo?"

Attention

I swam across the river to get to the other **bank**.
I walked across the road to get cash from the **bank**.



The word '**bank**' has different meanings in the two sentences. However, this can be detected only by looking at the context provided by other words in the sequence.

Some words are more important than others in determining the interpretation of 'bank'.

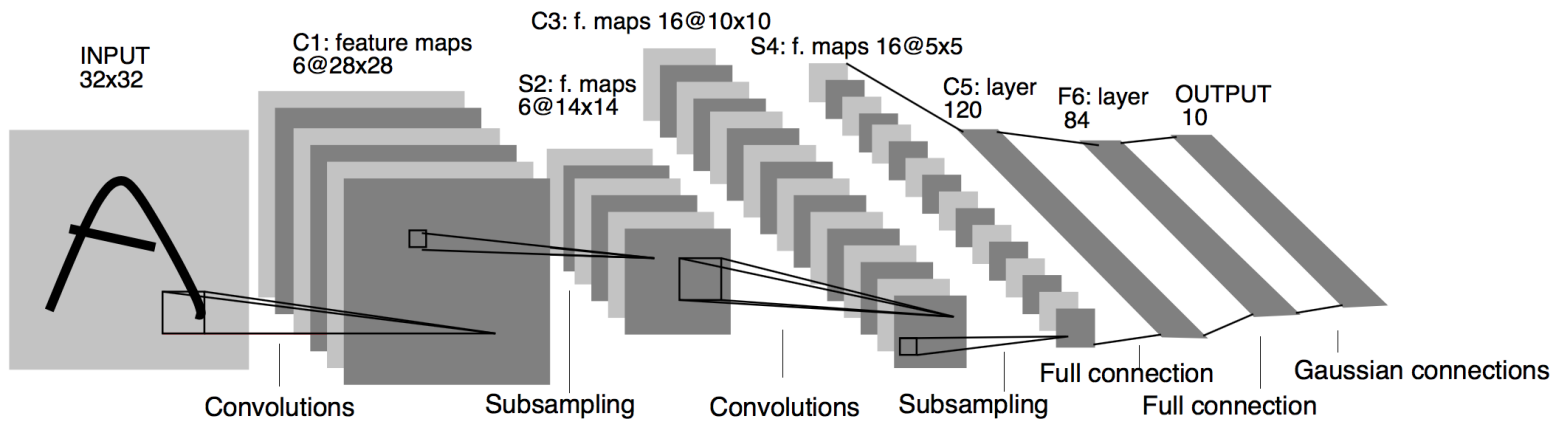
- In the first sentence, the words '**swam**' and '**river**' most strongly indicate that 'bank' refers to the side of a river
- In the second sentence, the word '**cash**' is a strong indicator that 'bank' refers to a financial institution.

We see that to determine the appropriate interpretation of 'bank', a neural network processing such a sentence should **attend** to, in other words rely more heavily on, specific words from the rest of the sequence.

Transformers

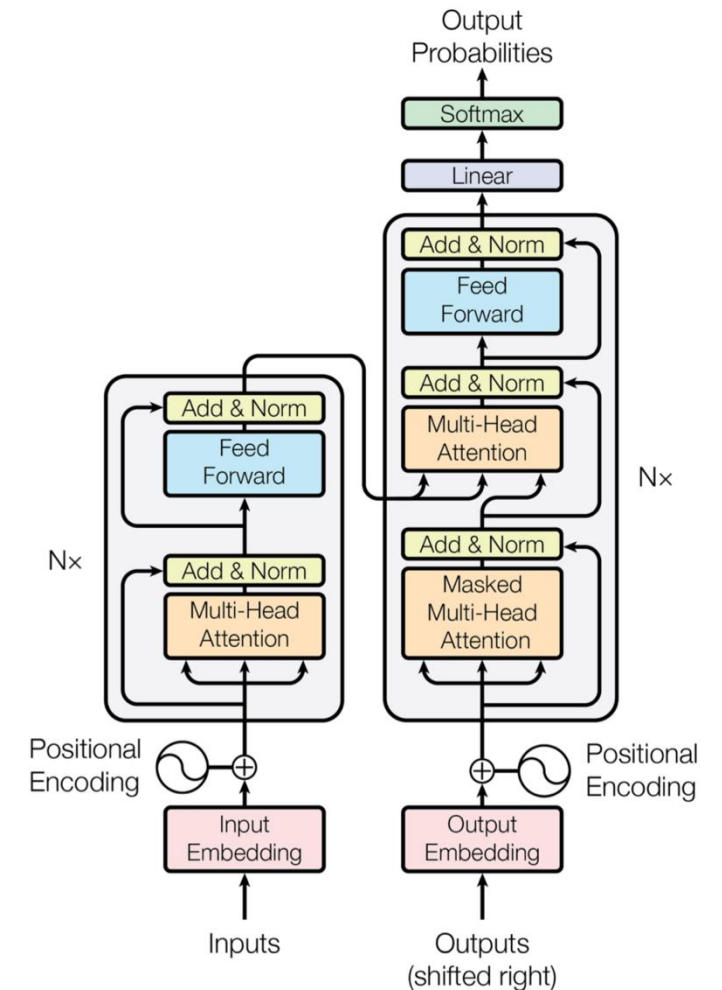
Background

- Convolutional neural networks (CNN) has been dominating
 - Greater scale
 - More extensive connections
 - More sophisticated forms of convolution



Background

- Transformers
 - Competitive alternative to CNN
 - A sequence-to-sequence model
 - Model of choice in natural language processing (NLP)
 - Notable for its use of **attention** to model long-range dependencies in data
 - Generally found to perform best in settings with large amounts of training data
 - Enable multi-modality learning



Background

Attention Is All You Need

CEO and co-founder at Essential AI

Ashish Vaswani*
Google Brain
avaswani@google.com

Founder of Character.AI

Noam Shazeer*
Google Brain
noam@google.com

Co-founder at Essential AI

Niki Parmar*
Google Research
nikip@google.com

CEO and co-founder of
Inceptive Nucleics

Jakob Uszkoreit*
Google Research
usz@google.com

Co-founder at Sakana AI

Llion Jones*
Google Research
llion@google.com

CEO and co-founder of Cohere

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain

OpenAI

lukaszkaizer@google.com

Co-founder at NEAR Protocol

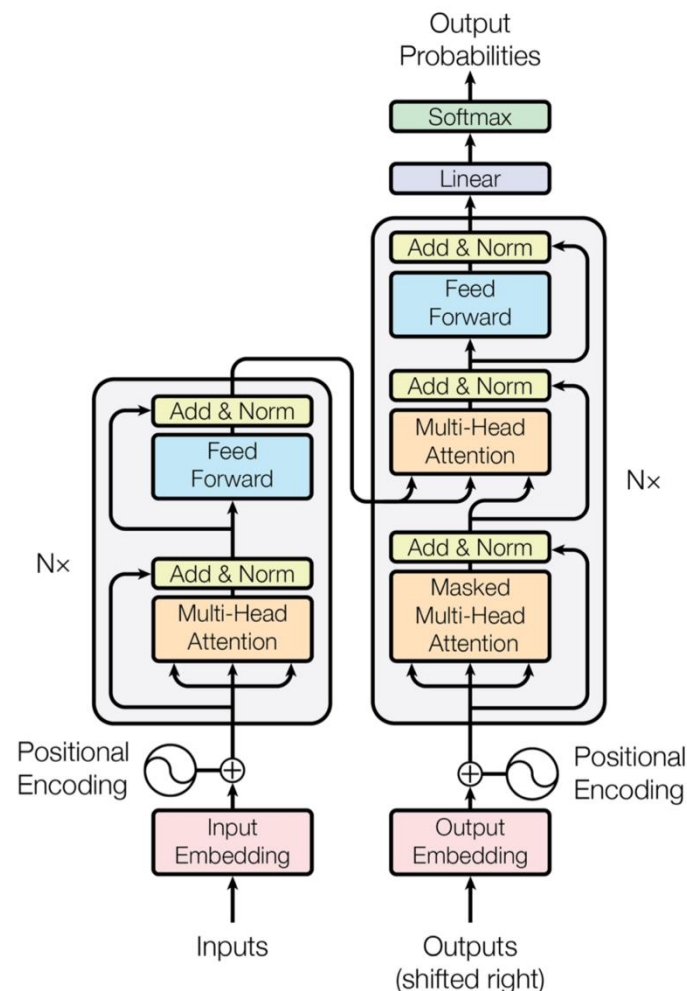
Illia Polosukhin* †
illia.polosukhin@gmail.com

Background

Like LSTM, Transformer is an architecture for transforming one sequence into another one with the help of two parts (Encoder and Decoder)

But it differs from the existing sequence-to-sequence models because it does not imply any Recurrent Networks (GRU, LSTM, etc.).

- During training, layer outputs can be calculated in parallel, instead of a series like an RNN
- Attention-based models allow modeling of dependencies without regard to their distance in the input or output sequences



Background

✦ GPT-4

GPT = Generative Pre-trained Transformer



What is the best way to learn knowledge about Transformers (a machine learning model)?



Transformers are an important concept in modern machine learning, especially in the field of natural language processing. They are the key model behind the technology in GPT-3 and GPT-4, BERT, T5, and many other influential architectures.



Here is a suggested path for learning about Transformers:

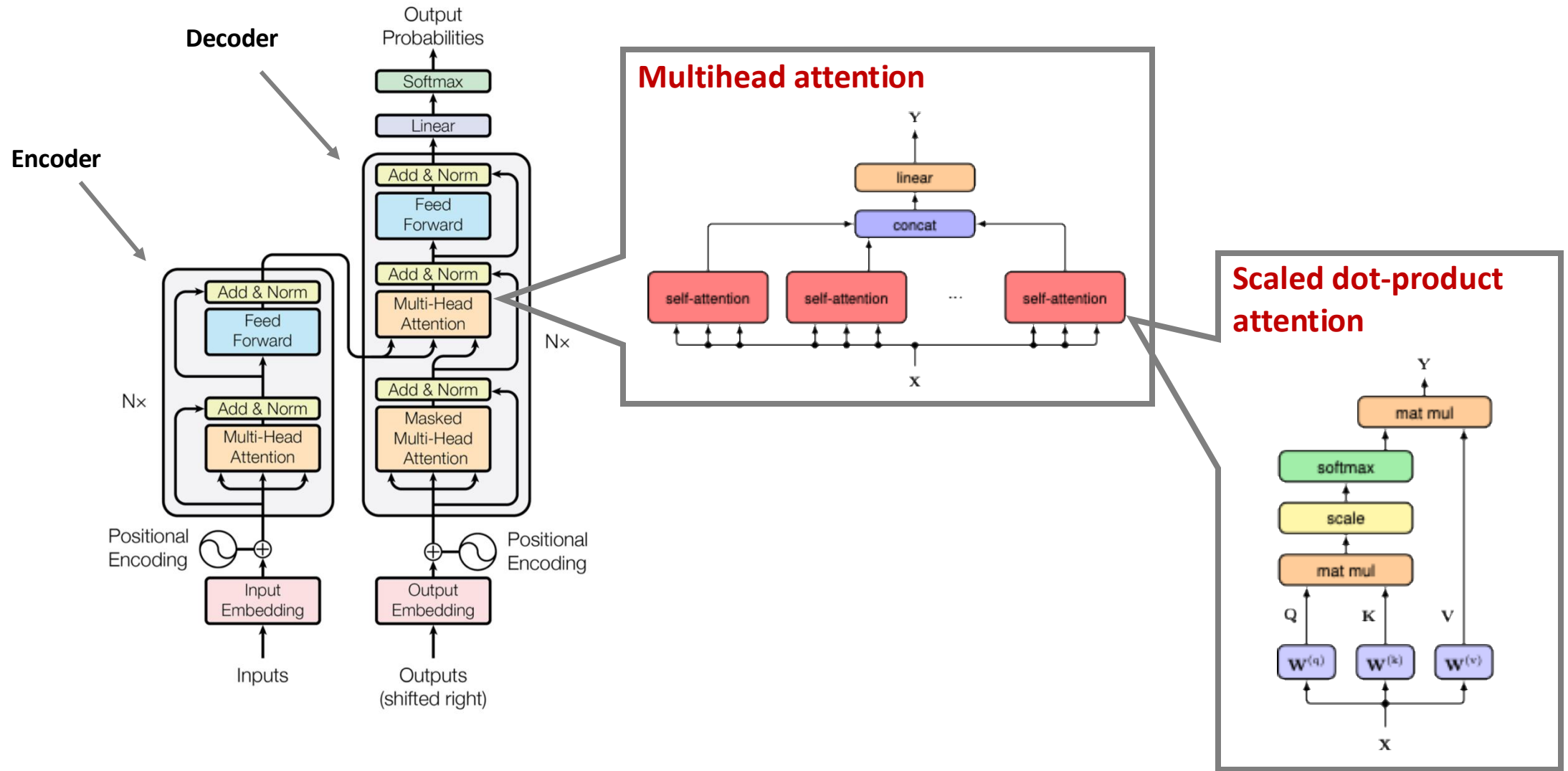
1. Understand the Basics of Machine Learning and Deep Learning

Before you get into Transformers, it's crucial to have a solid understanding of Machine Learning and Deep Learning. Knowing the fundamental concepts of Deep Learning such as neural networks, activation functions, loss functions, backpropagation, convolutional neural networks (CNNs), recurrent neural networks (RNNs), and Long Short Term Memory (LSTM) units is very helpful.

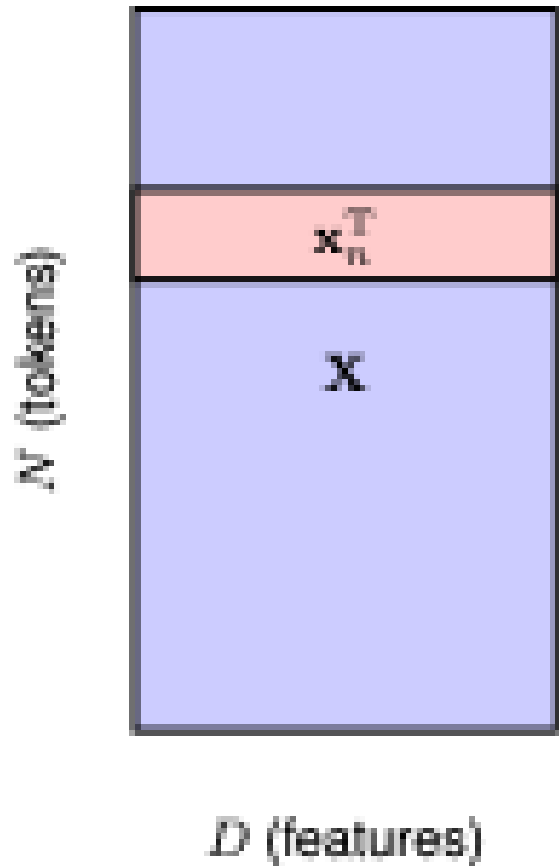
Background



Transformers overview



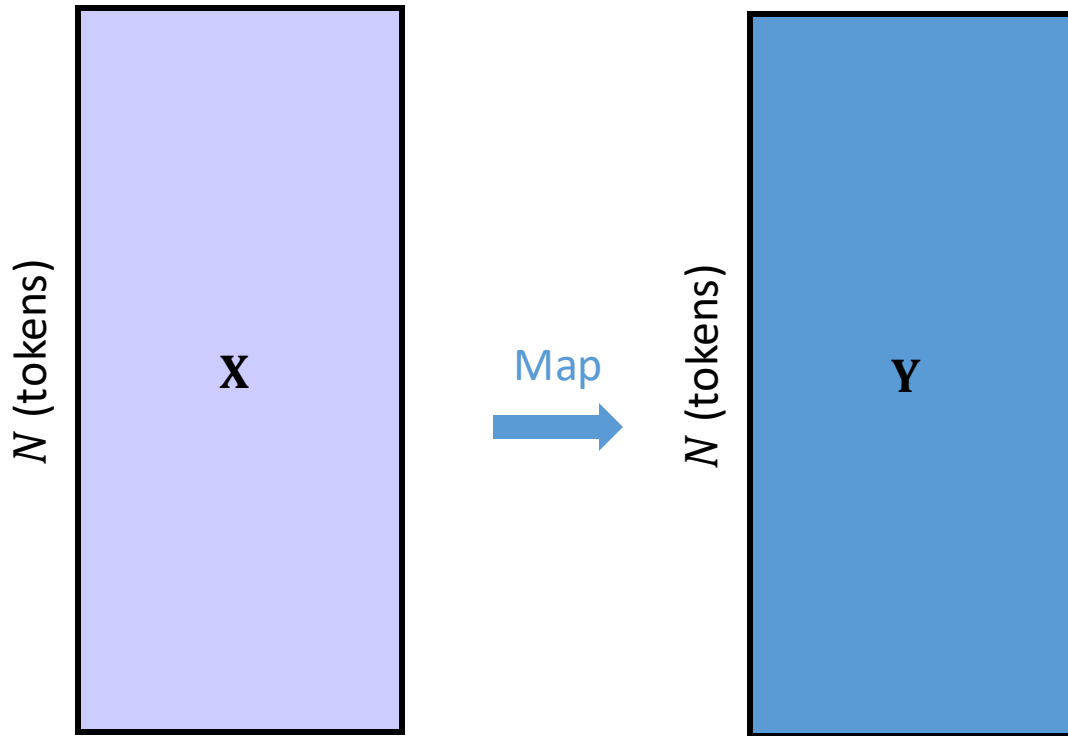
Tokens



- **Token** = the input data to a transformer, a set of vectors $\{\mathbf{x}_n\}$ of dimensionality D , where $n = 1, \dots, N$
- A **token** might, for example, correspond to a word within a sentence, a patch within an image, or an amino acid within a protein.
- A powerful property of transformers is that we do not have to design a new neural network architecture to handle a mix of different data types but instead can simply combine the data variables into a joint set of tokens

Attention coefficients

Attention is a mechanism that a model can learn to make predictions by selectively attending to a given set of data. The amount of attention is **quantified by learned weights** and thus the output is usually formed as a **weighted average**.

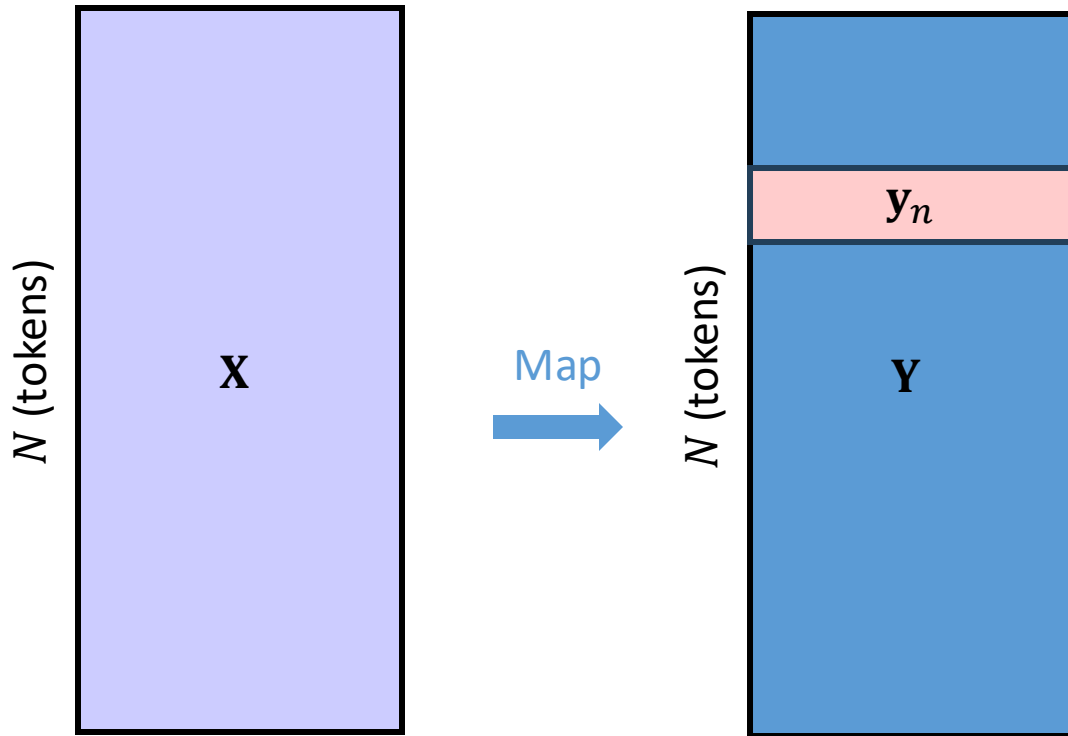


Example

Suppose that we have a set of input tokens $\mathbf{x}_1, \dots, \mathbf{x}_N$ in an embedding space

We want to map this to another set $\mathbf{y}_1, \dots, \mathbf{y}_N$ having the same number of tokens but in a new embedding space that captures a richer semantic context.

Attention coefficients



Consider a particular output token $\mathbf{y}_n$

A simple way to achieve the mapping is to define each output vector $\mathbf{y}_n$ to be a linear combination of the input vectors $\mathbf{x}_1, \dots, \mathbf{x}_N$ with **attention weights** a_{nm}

$$\mathbf{y}_n = \sum_{m=1}^N a_{nm} \mathbf{x}_m$$

The output is a blend of the input vectors with some inputs given more weight than others.

The attention weights must satisfy the following two constraints

$$a_{nm} \geq 0$$

$$\sum_{m=1}^N a_{nm} = 1.$$

The next question is how to determine the **attention weights**?

Self-attention

Key, **Value**, **Query** (analogy with information retrieval)

Choose which movie to watch in an online movie streaming service.

Each movie is associated with a list of attributes, such as the genre (comedy, action, etc.), the names of the leading actors, the length of the movie, and so on. The user could then search through a catalogue to find a movie that matches their preferences.

The corresponding movie file itself is called a **value**.

We could automate this by encoding the attributes of each movie in a vector called the **key**.

The user could then provide their own personal vector of values for the desired attributes, which we call the **query**.

value



key

Comedy
Romantic
Cartoon

Action
Sci-fi

Horror
Mystery

query

I want to watch an *action* movie tonight

Self-attention

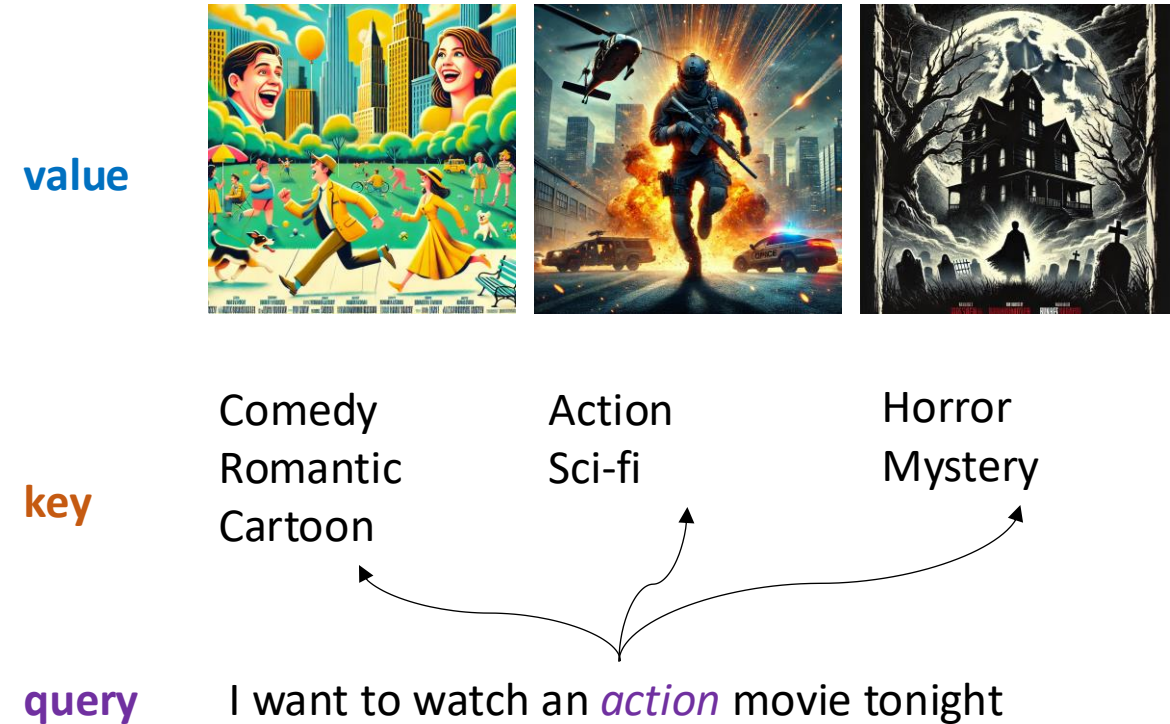
Key, Value, Query

The movie service could then compare the **query** vector with all the **key** vectors to find the best match and send the corresponding movie to the user in the form of the **value** file.

We can think of the user 'attending' to the particular movie whose **key** most closely matches their **query**.

This would be considered a form of hard attention in which a single **value** vector is returned.

For the transformer, we generalize this to soft attention in which we use continuous variables to measure the degree of match between queries and keys and we then use these variables to weight the influence of the value vectors on the outputs.

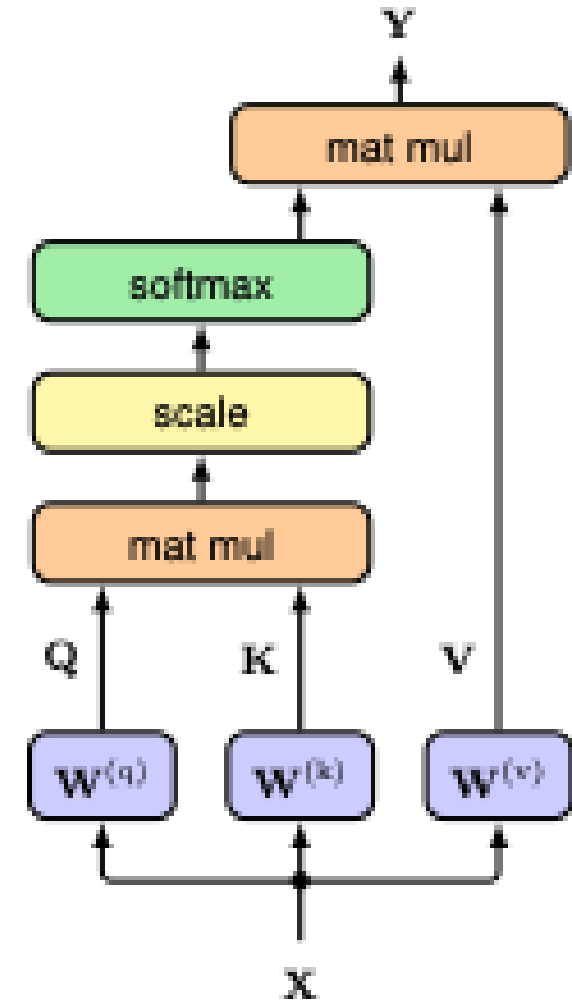


Self-attention

Self-attention = Scaled dot-product attention

The output is a weighted sum of the values, where the weight assigned to each value is determined by the dot-product of the query with all the keys

$$\mathbf{Y} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \equiv \text{SoftMax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{D_k}}\right)\mathbf{V}$$



Information flow in a scaled dot-product self-attention. This structure constitutes a single attention 'head'.

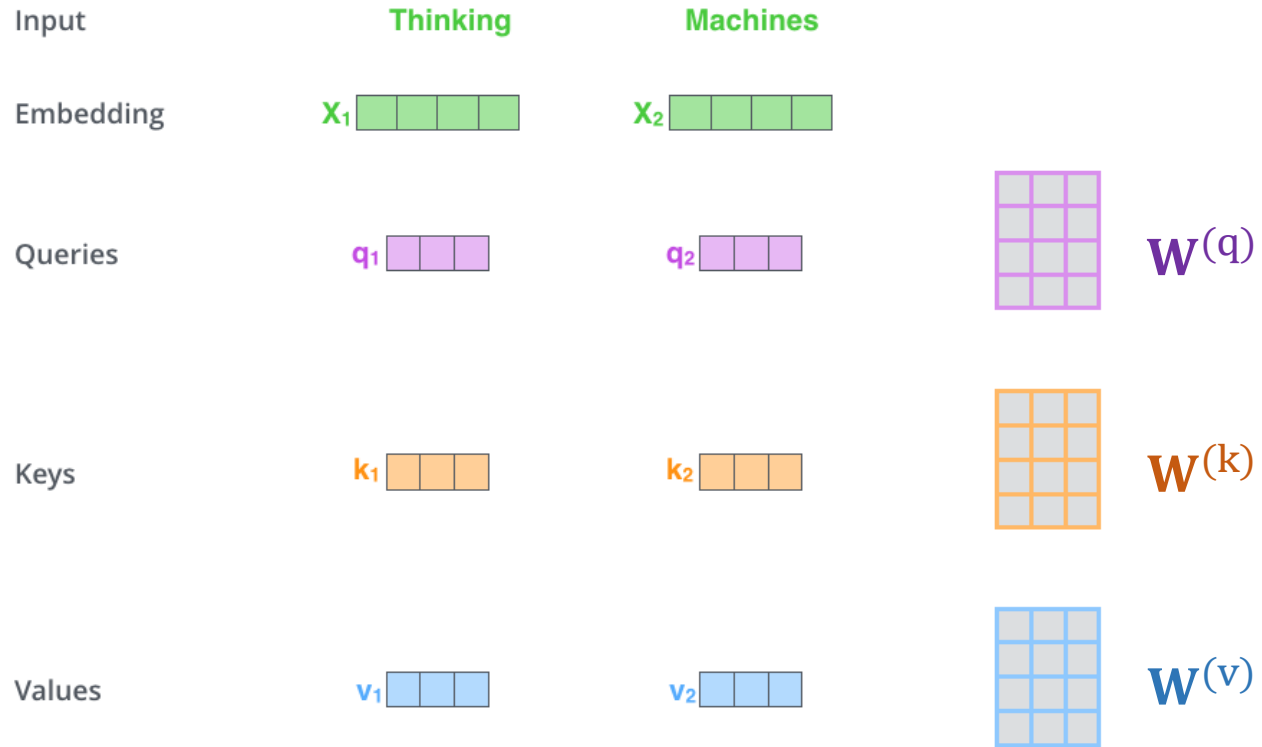
Self-attention in detail

First Step

Create three vectors from each of the encoder's input vectors (in this case, the **embedding** of each word).

So for each word, we create a **Query vector**, a **Key vector**, and a **Value vector**.

These vectors are created by multiplying the embedding by three matrices that we trained during the training process.



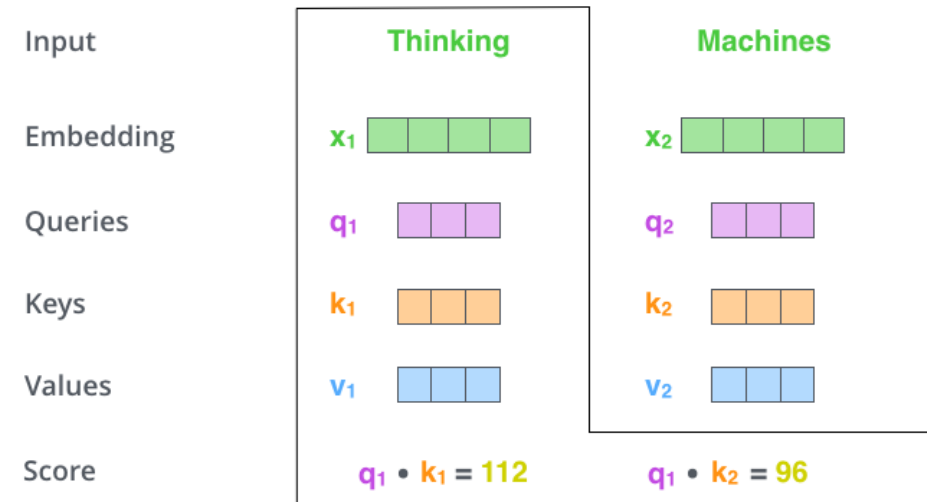
Self-attention in detail

Second Step

Calculate a score for each word of the input sentence against a word.

The score determines how much focus to place on other parts of the input sentence as we encode a word at a certain position.

The score is calculated by taking the **dot product** of the **query vector** with the **key vector** of the respective word we're scoring.



Self-attention in detail

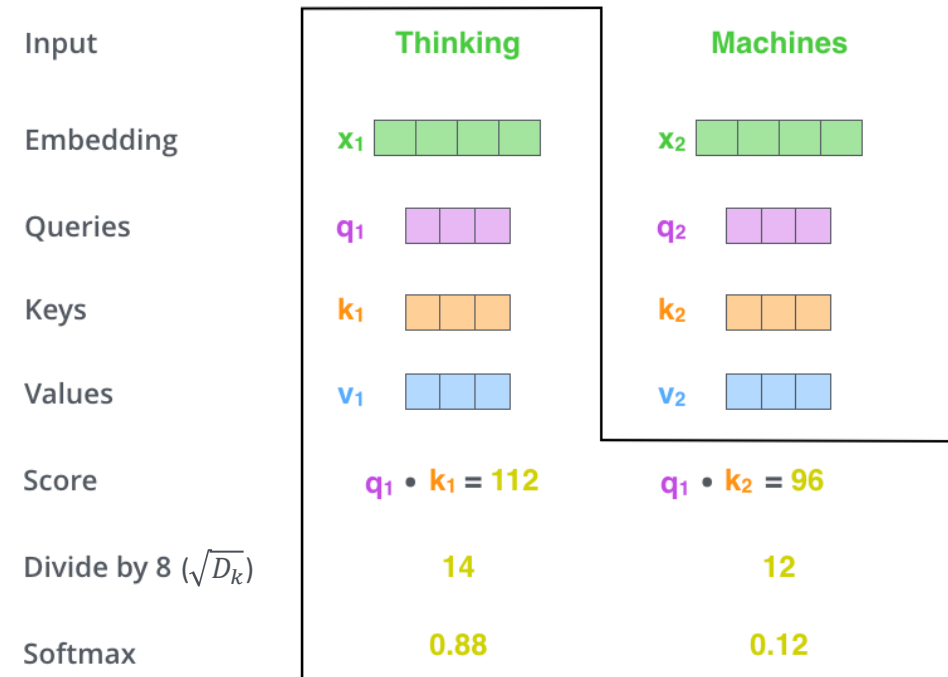
Third Step

Divide the scores by $\sqrt{D_k}$, the square root of the dimension of the key vectors

Why dividing by $\sqrt{D_k}$?

Fourth Step

Softmax normalizes the scores so they're all positive and add up to 1



This Softmax score determines how much each word will be expressed at this position. Clearly the word at this position will have the highest Softmax score, but sometimes it's useful to attend to another word that is relevant to the current word.

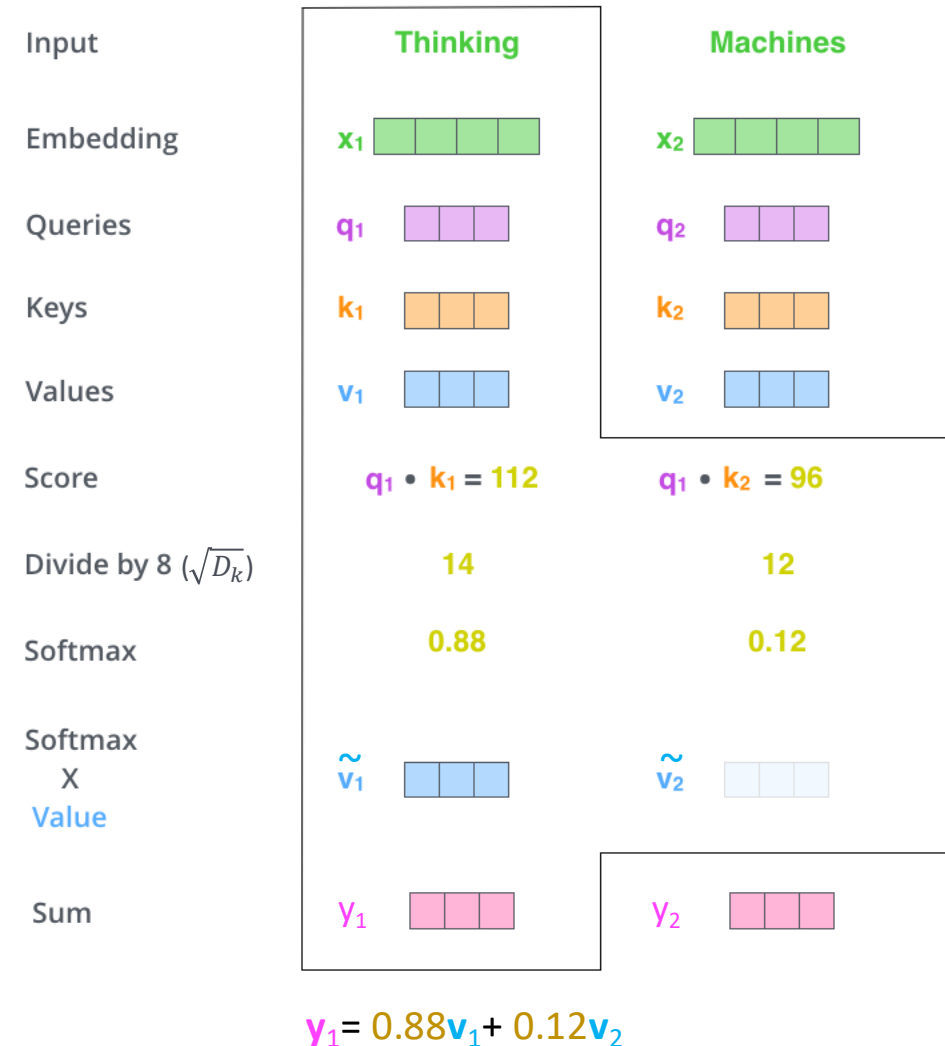
Self-attention in detail

Fifth Step

Multiply each value vector by the softmax score

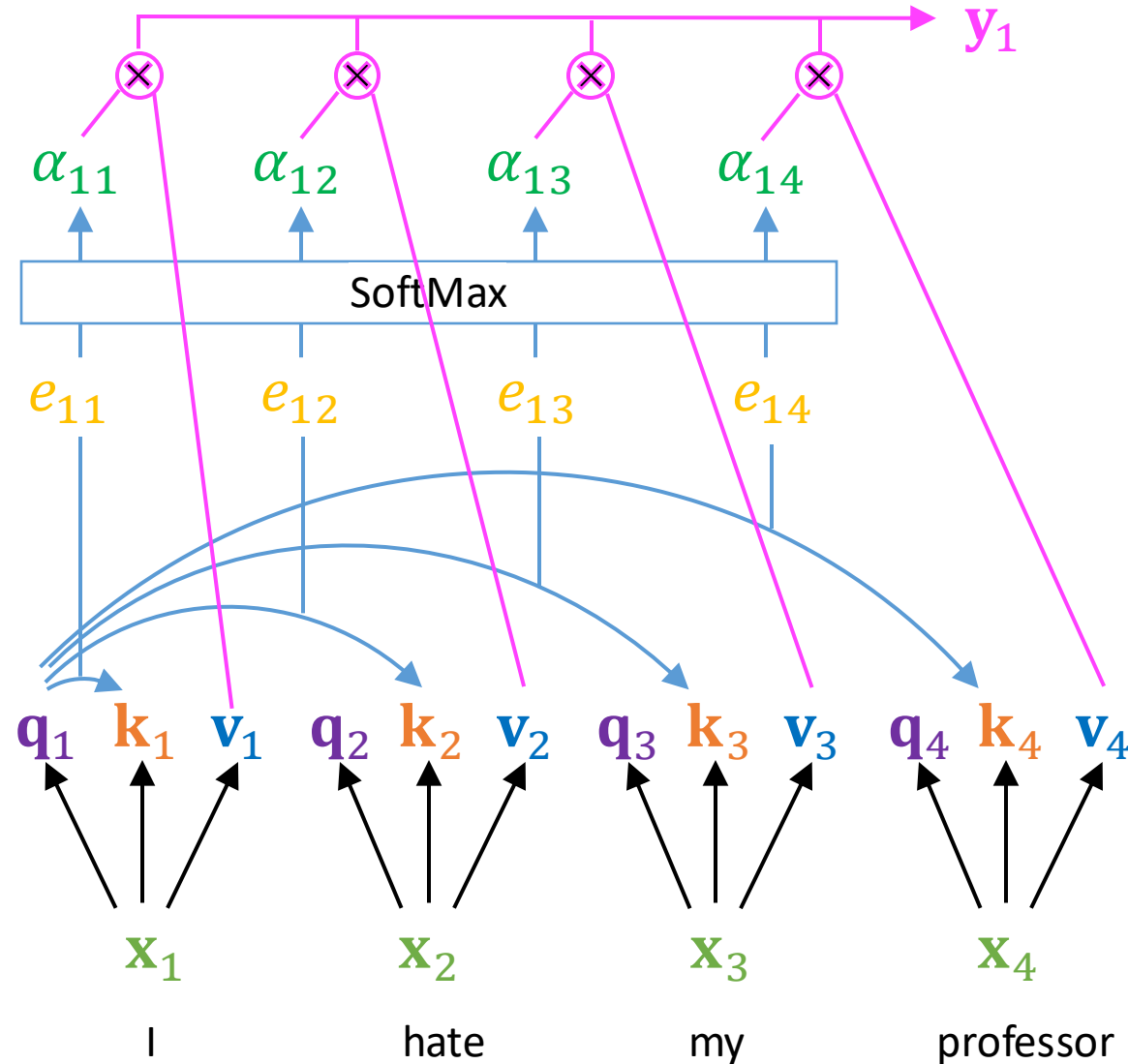
Sixth Step

Sum up the weighted value vectors to get the output of the self-attention layer at this position (for the first word)



Self-attention in detail

$$e = \frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{D_k}}$$



$$\mathbf{q} = \mathbf{x}\mathbf{W}^{(q)}$$

$$\mathbf{k} = \mathbf{x}\mathbf{W}^{(k)}$$

$$\mathbf{v} = \mathbf{x}\mathbf{W}^{(v)}$$

Matrix calculation of self-attention

First Step

Calculate the Query, Key, and Value matrices.

Pack our embeddings into a matrix $\mathbf{X}$, and multiplying it by the weight matrices we've trained ($\mathbf{W}^{(q)}$, $\mathbf{W}^{(k)}$, $\mathbf{W}^{(v)}$)

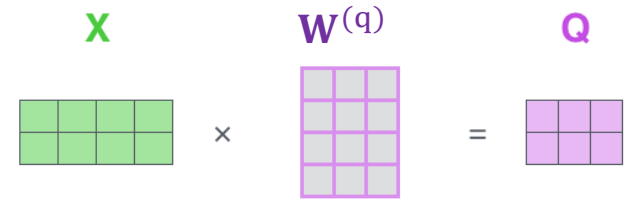
Second Step

Calculate the outputs of the self-attention layer.

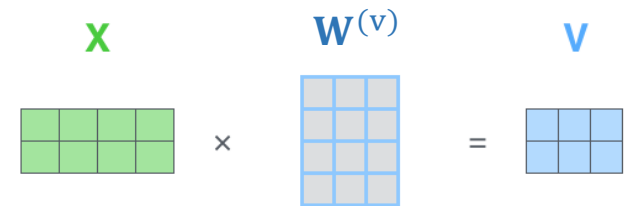
SoftMax is **row-wise**

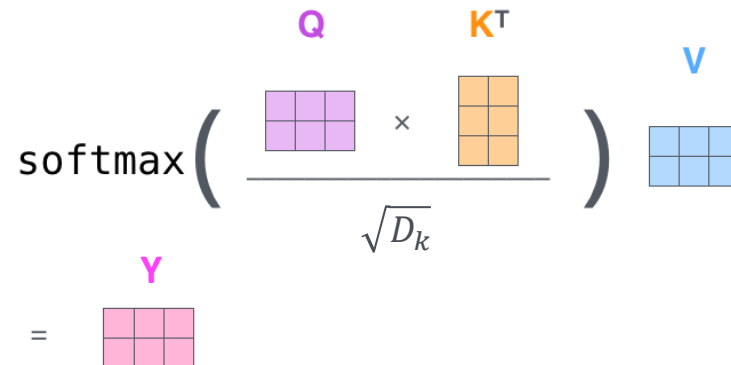
$$\mathbf{Y} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \equiv \text{SoftMax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{D_k}}\right)\mathbf{V}$$

Every row in the $\mathbf{X}$ matrix corresponds to a token in the input sentence.

$$\mathbf{X} \times \mathbf{W}^{(q)} = \mathbf{Q}$$


$$\mathbf{X} \times \mathbf{W}^{(k)} = \mathbf{K}$$


$$\mathbf{X} \times \mathbf{W}^{(v)} = \mathbf{V}$$


$$\text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{D_k}}\right)\mathbf{V}$$


Example

Assuming we have two word embeddings:

(1, 2, 3)

(4, 5, 6)

And the given $\mathbf{W}^{(q)}$, $\mathbf{W}^{(k)}$, $\mathbf{W}^{(v)}$ matrices are respectively given as

$$\begin{pmatrix} 0.01 & 0.03 \\ 0.02 & 0.02 \\ 0.03 & 0.01 \end{pmatrix}, \begin{pmatrix} 0.05 & 0.05 \\ 0.06 & 0.05 \\ 0.07 & 0.05 \end{pmatrix}, \begin{pmatrix} 0.02 & 0.02 \\ 0.01 & 0.02 \\ 0.01 & 0.01 \end{pmatrix}$$

Compute the output of the scaled-dot product attention, $\mathbf{Y} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$.

Example

Step 1: Find **Q**, **K**, **V**

$$\mathbf{Q} = \mathbf{XW}^{(q)} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \begin{pmatrix} 0.01 & 0.03 \\ 0.02 & 0.02 \\ 0.03 & 0.01 \end{pmatrix} = \begin{pmatrix} 0.14 & 0.10 \\ 0.32 & 0.28 \end{pmatrix}$$

$$\mathbf{K} = \mathbf{XW}^{(k)} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \begin{pmatrix} 0.5 & 0.5 \\ 0.6 & 0.5 \\ 0.7 & 0.5 \end{pmatrix} = \begin{pmatrix} 0.38 & 0.30 \\ 0.92 & 0.75 \end{pmatrix}$$

$$\mathbf{V} = \mathbf{XW}^{(v)} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \begin{pmatrix} 0.02 & 0.02 \\ 0.01 & 0.02 \\ 0.01 & 0.01 \end{pmatrix} = \begin{pmatrix} 0.07 & 0.09 \\ 0.19 & 0.24 \end{pmatrix}$$

Example

Step 2: Find $\mathbf{Y} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{D_k}}\right) \mathbf{V}$$

Perform row-wise SoftMax

$$\text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{D_k}}\right) = \text{Softmax}\left(\frac{\begin{pmatrix} 0.14 & 0.10 \\ 0.32 & 0.28 \end{pmatrix} \begin{pmatrix} 0.38 & 0.92 \\ 0.30 & 0.75 \end{pmatrix}}{\sqrt{2}}\right) = \text{Softmax}\begin{pmatrix} 0.0588 & 0.1441 \\ 0.1454 & 0.3566 \end{pmatrix} = \begin{pmatrix} 0.4787 & 0.5213 \\ 0.4474 & 0.5526 \end{pmatrix}$$

$$\text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{D_k}}\right) \mathbf{V} = \begin{pmatrix} 0.4787 & 0.5213 \\ 0.4474 & 0.5526 \end{pmatrix} \begin{pmatrix} 0.07 & 0.09 \\ 0.19 & 0.24 \end{pmatrix} = \begin{pmatrix} 0.1326 & 0.1682 \\ 0.1363 & 0.1729 \end{pmatrix}$$

$$\frac{e^{z_j}}{\sum_j e^{z_j}} = \frac{e^{0.0588}}{e^{0.0588} + e^{0.1441}} = 0.4787$$



Summary

Algorithm 12.1: Scaled dot-product self-attention

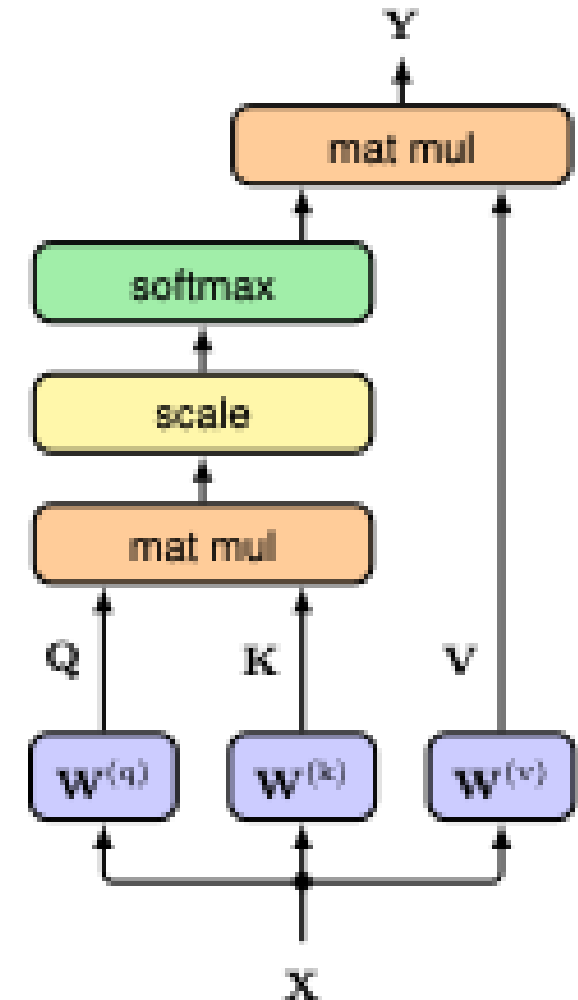
Input: Set of tokens $\mathbf{X} \in \mathbb{R}^{N \times D} : \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$
Weight matrices $\{\mathbf{W}^{(q)}, \mathbf{W}^{(k)}\} \in \mathbb{R}^{D \times D_k}$ and $\mathbf{W}^{(v)} \in \mathbb{R}^{D \times D_v}$
Output: $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \in \mathbb{R}^{N \times D_v} : \{\mathbf{y}_1, \dots, \mathbf{y}_N\}$

$\mathbf{Q} = \mathbf{XW}^{(q)}$ // compute queries $\mathbf{Q} \in \mathbb{R}^{N \times D_k}$

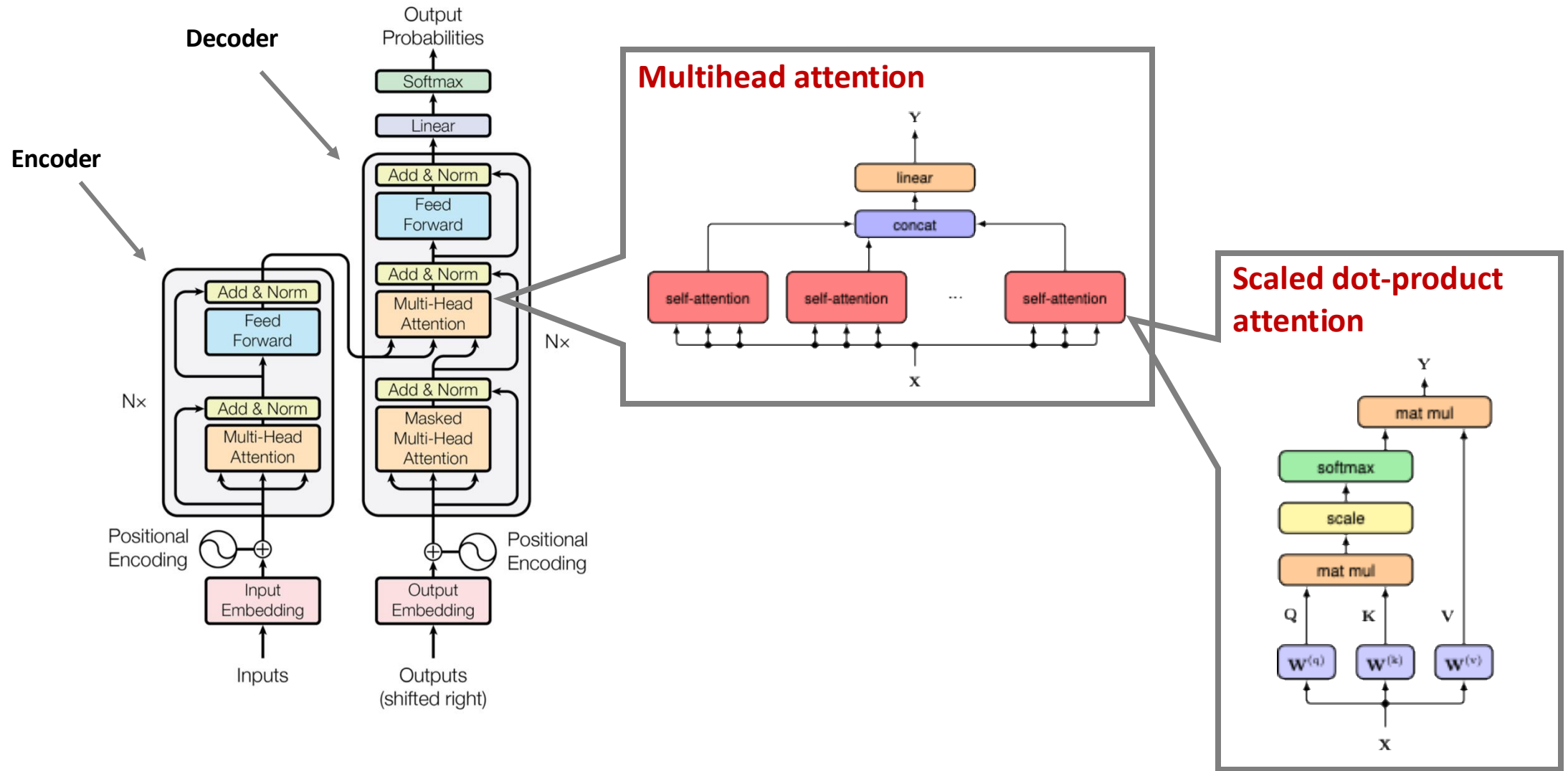
$\mathbf{K} = \mathbf{XW}^{(k)}$ // compute keys $\mathbf{K} \in \mathbb{R}^{N \times D_k}$

$\mathbf{V} = \mathbf{XW}^{(v)}$ // compute values $\mathbf{V} \in \mathbb{R}^{N \times D_v}$

return $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left[\frac{\mathbf{QK}^T}{\sqrt{D_k}}\right] \mathbf{V}$



Transformers overview



Multi-head self-attention



Imagine you are reading a story

"Alice looked out the window and saw the sun setting over the mountains. She felt a cool breeze and decided it was time to head home."

When you read this story, you naturally pay attention to different aspects to fully understand it:

- Visual Imagery: You might picture the sun setting over the mountains.
- Emotions: You sense Alice's feelings—perhaps she feels calm or reflective.
- Actions: You note that Alice decides to head home.

Multi-head self-attention



Single-Head Attention (Single Focus):

Imagine trying to consider all these aspects at once with just one focus. You might get a general understanding but miss specific details in each aspect.

Multi-Head Attention (Multiple Focuses):

Now, imagine you have multiple spotlights:

- One spotlight highlights visual elements (sun setting, mountains).
- Another spotlight highlights emotional cues (felt a cool breeze, decided it was time).
- A third spotlight focuses on actions (looked out the window, head home).

Multi-head self-attention

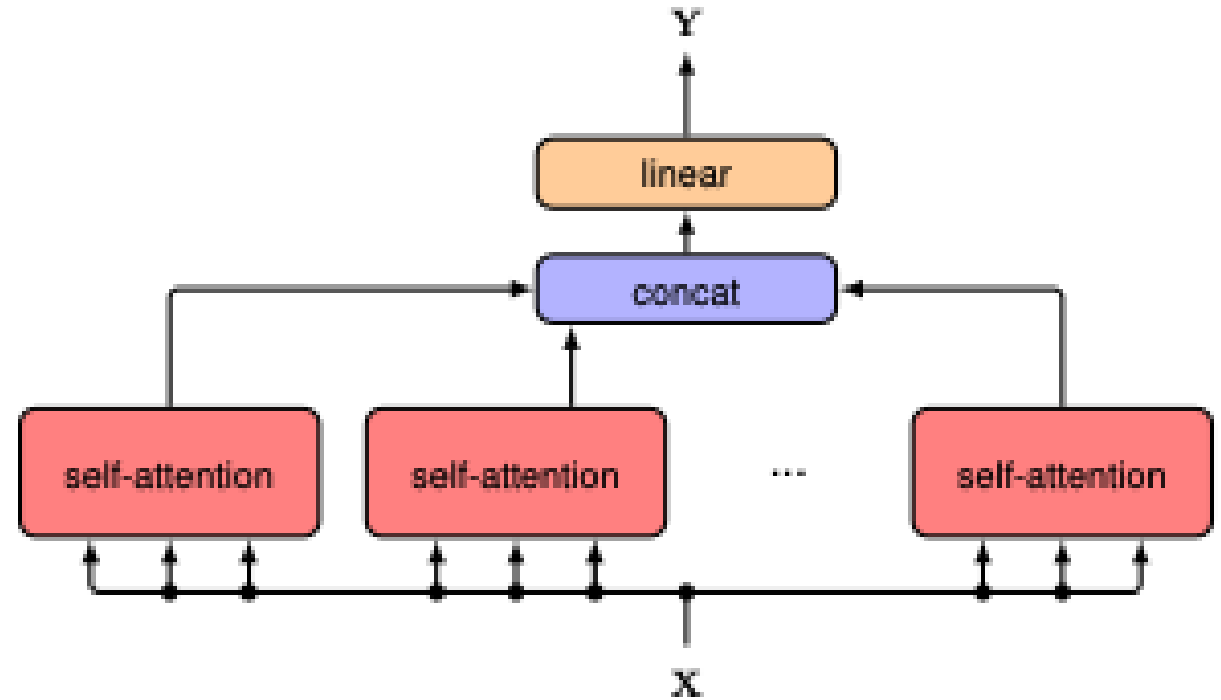
Multi-Head Self-Attention

There might be multiple patterns of attention that are relevant at the same time.

Using a single attention head can lead to **averaging over** these effects.

Instead we can use multiple attention heads in parallel.

These consist of **identically structured copies of the single head**, with **independent learnable parameters** that govern the calculation of the query, key, and value matrices.



Multi-head self-attention

Suppose we have H heads indexed by $h = 1, \dots, H$ of the form

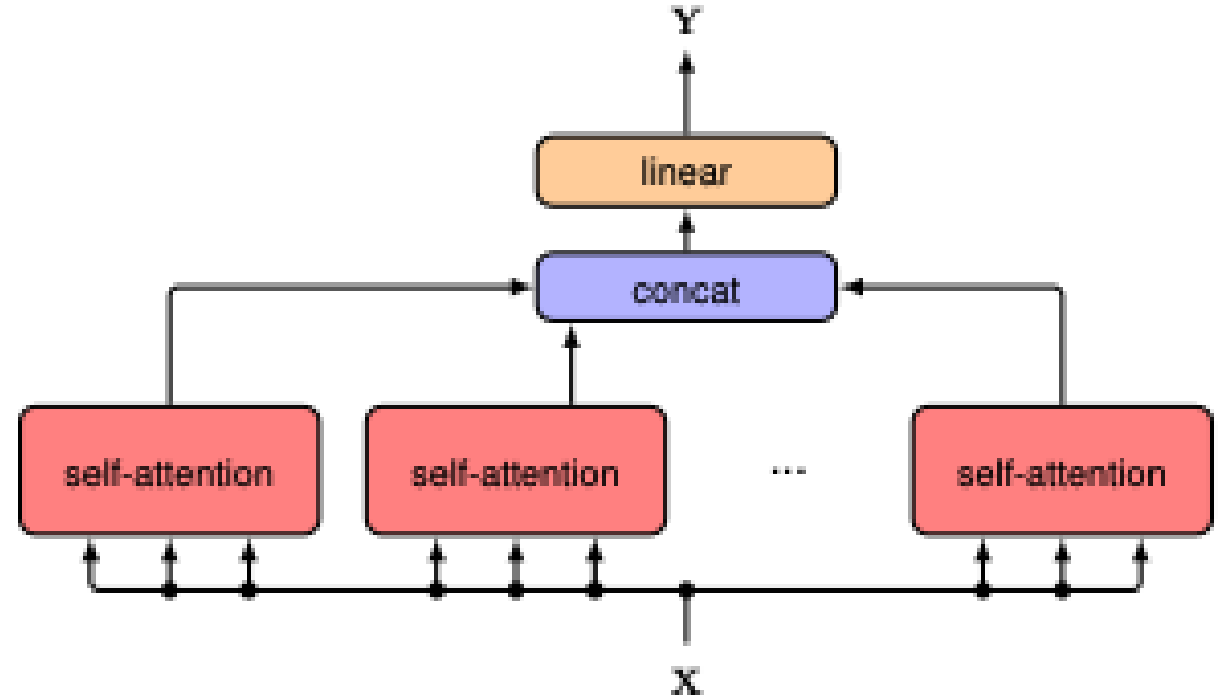
$$\mathbf{H}_h = \text{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h)$$

We define separate query, key, and value matrices for each head using

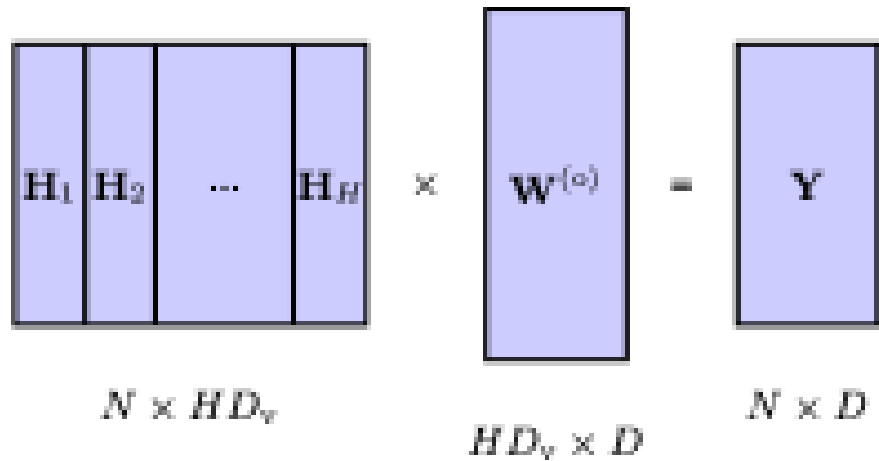
$$\mathbf{Q}_h = \mathbf{XW}_h^{(q)}$$

$$\mathbf{K}_h = \mathbf{XW}_h^{(k)}$$

$$\mathbf{V}_h = \mathbf{XW}_h^{(v)}$$



Multi-head self-attention

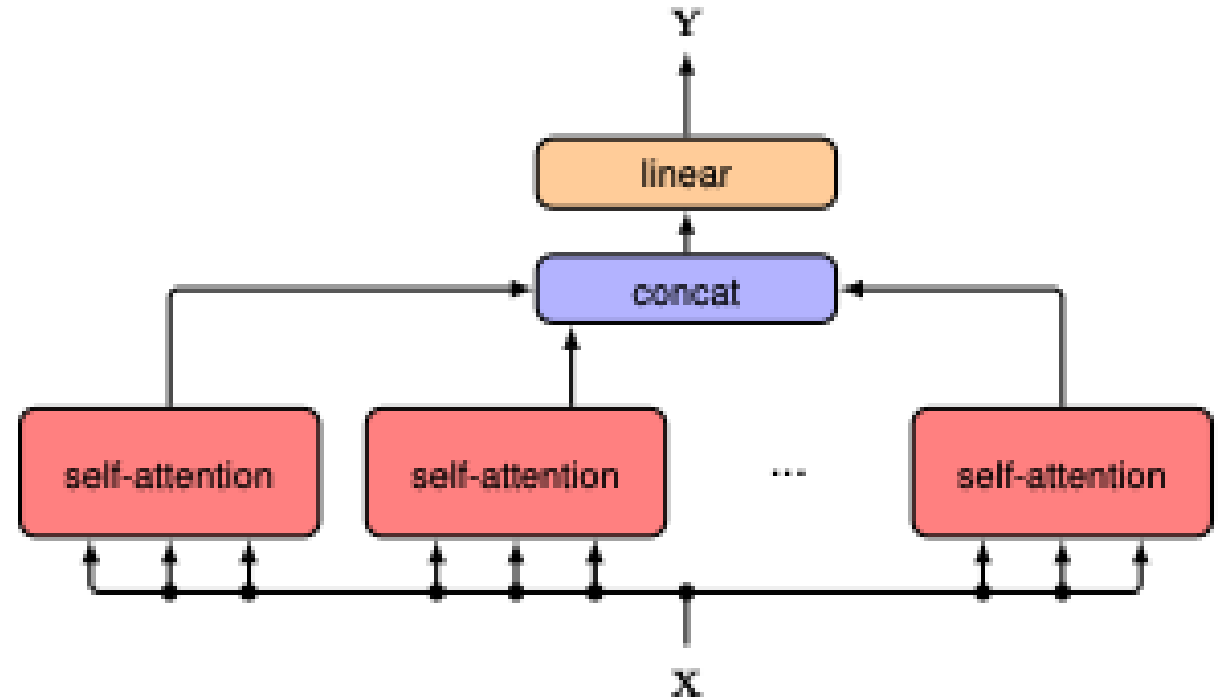


The heads are first concatenated into a single matrix

The result is then linearly transformed using a matrix $W^{(o)}$ to give a combined output in the form

$$Y(X) = \text{Concat}[H_1, \dots, H_H] W^{(o)}$$

The elements of the matrix $W^{(o)}$ are learned during the training phase along with the query, key, and value matrices.



Summary

Algorithm 12.2: Multi-head attention

Input: Set of tokens $\mathbf{X} \in \mathbb{R}^{N \times D} : \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$
Query weight matrices $\{\mathbf{W}_1^{(q)}, \dots, \mathbf{W}_H^{(q)}\} \in \mathbb{R}^{D \times D}$
Key weight matrices $\{\mathbf{W}_1^{(k)}, \dots, \mathbf{W}_H^{(k)}\} \in \mathbb{R}^{D \times D}$
Value weight matrices $\{\mathbf{W}_1^{(v)}, \dots, \mathbf{W}_H^{(v)}\} \in \mathbb{R}^{D \times D_v}$
Output weight matrix $\mathbf{W}^{(o)} \in \mathbb{R}^{HD_v \times D}$
Output: $\mathbf{Y} \in \mathbb{R}^{N \times D} : \{\mathbf{y}_1, \dots, \mathbf{y}_N\}$

// compute self-attention for each head (Algorithm 12.1)

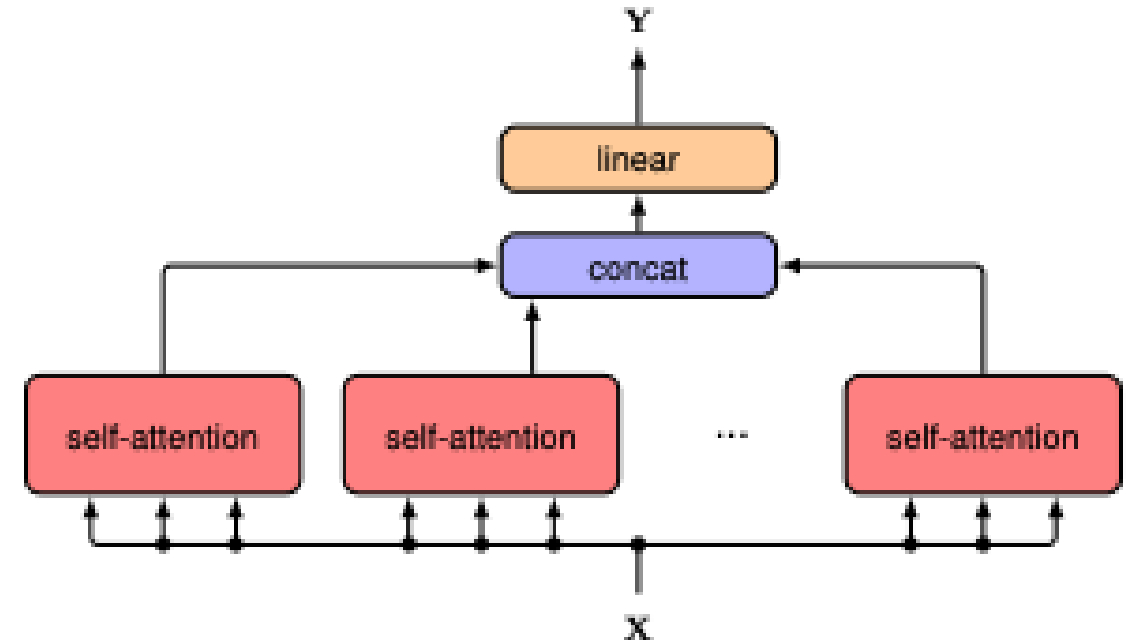
for $h = 1, \dots, H$ do

$\mathbf{Q}_h = \mathbf{XW}_h^{(q)}$, $\mathbf{K}_h = \mathbf{XW}_h^{(k)}$, $\mathbf{V}_h = \mathbf{XW}_h^{(v)}$
 $\mathbf{H}_h = \text{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h)$ // $\mathbf{H}_h \in \mathbb{R}^{N \times D_v}$

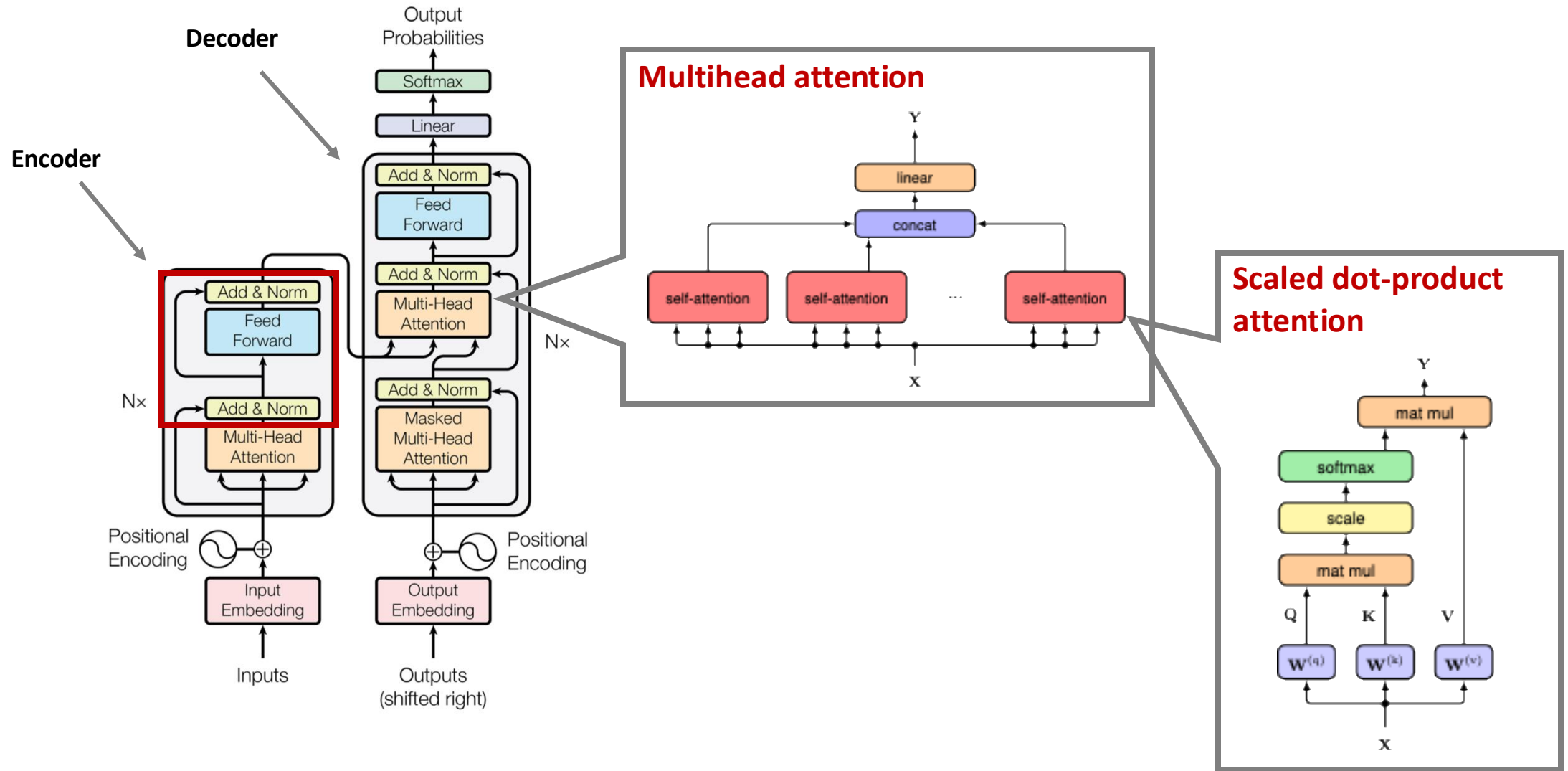
end for

$\mathbf{H} = \text{Concat}[\mathbf{H}_1, \dots, \mathbf{H}_N]$ // concatenate heads

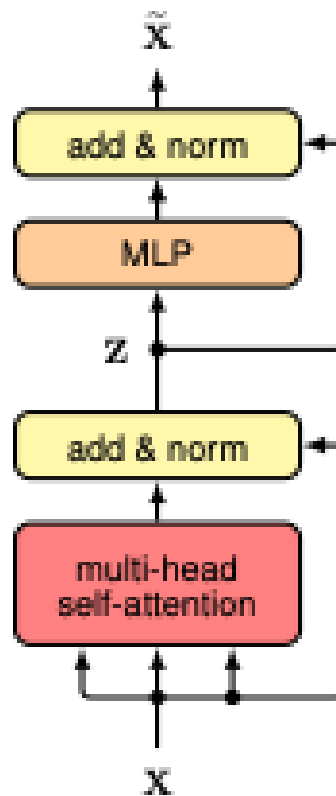
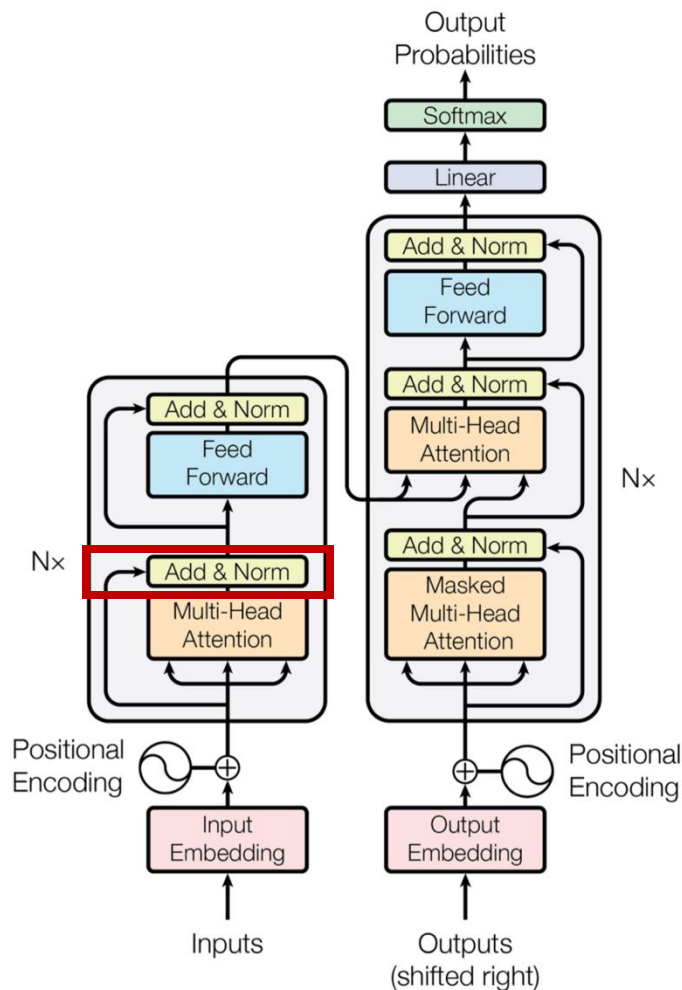
return $\mathbf{Y}(\mathbf{X}) = \mathbf{HW}^{(o)}$



Transformers overview



Transformation layers



Neural networks benefit greatly from depth, and so we would like to stack multiple self-attention layers on top of each other.

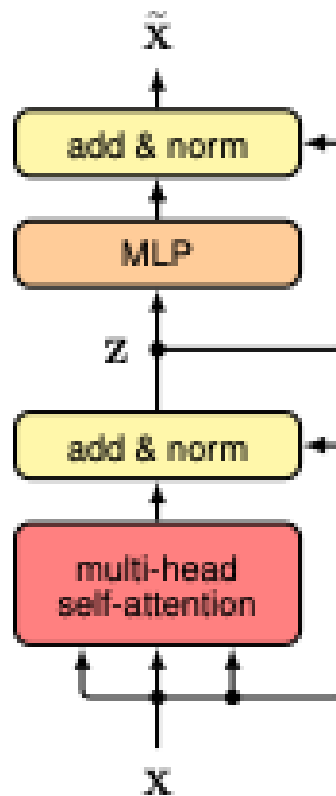
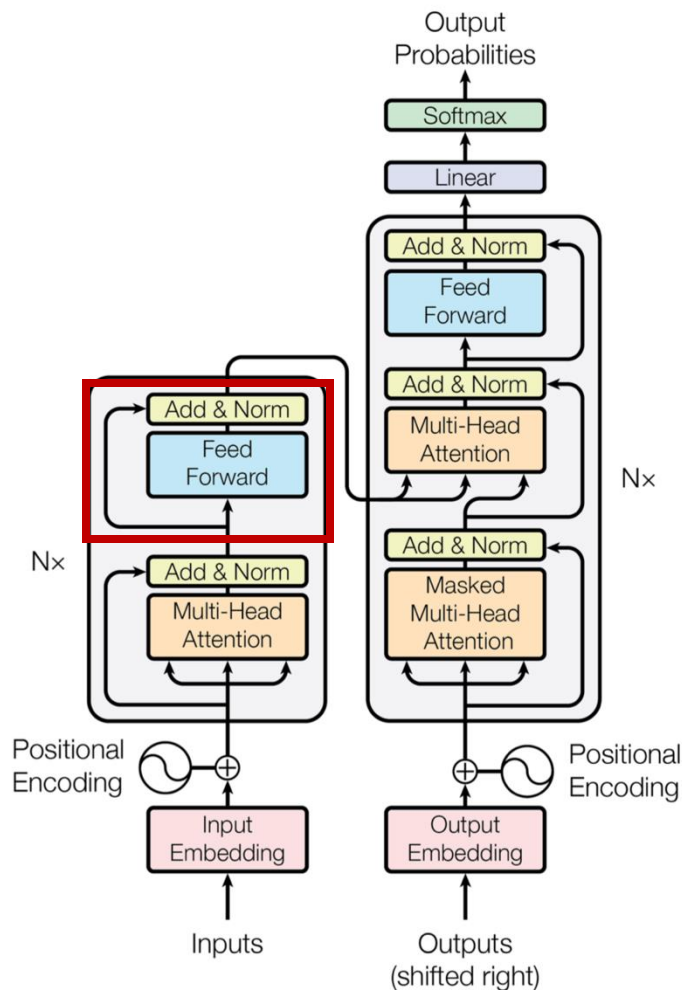
To improve training efficiency, we can introduce residual connections that bypass the multi-head structure.

The resulting transformation can be written as

$$Z = \text{LayerNorm} [Y(X) + X]$$

where Y is the multi-head self-attention

Transformation layers

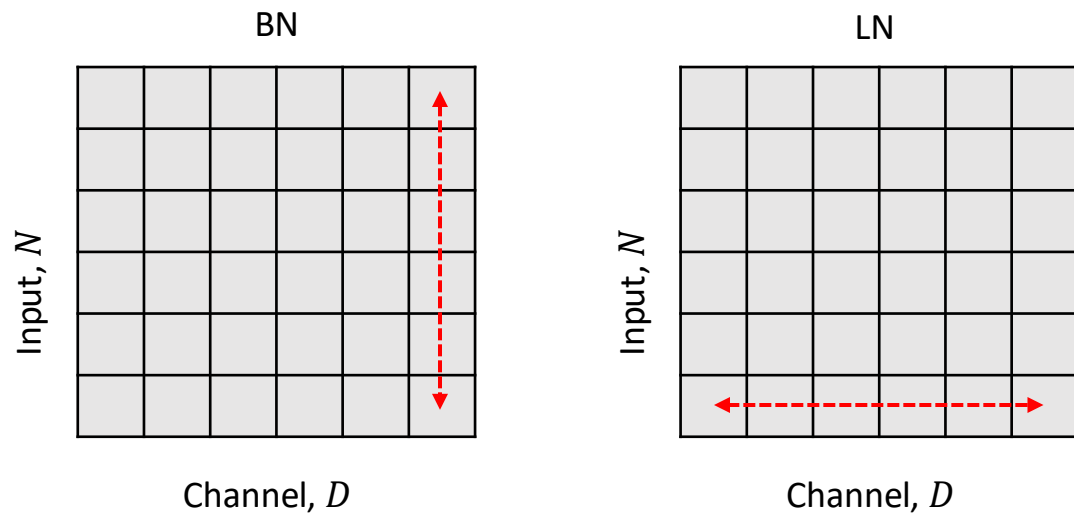


We can enhance the flexibility of the transformer by post-processing the output of each layer using a standard nonlinear neural network, denoted $\text{MLP}[\cdot]$

$$\tilde{\mathbf{X}} = \text{LayerNorm} [\text{MLP}[\mathbf{Z}] + \mathbf{Z}]$$

The same shared network is applied to each of the output vectors, corresponding to the rows of $\mathbf{Z}$, so that the transformer can process sequences of variable length

Transformation layers



Layer Normalization (LN) ¹

- The pixels along the red arrow are normalized by the same mean and variance, computed by aggregating the values of these pixels.
- BN is found unstable in Transformers ²
- Works well with RNN and now being used in Transformers

¹ Jimmy Lei Ba, Jamie Ryan Kiros, Geoffrey E. Hinton, [Layer Normalization](#), arXiv:1607.06450

² Sheng Shen, Zhewei Yao, Amir Gholami, Michael Mahoney, Kurt Keutzer, [Rethinking Batch Normalization in Transformers](#), ICML 2020

Summary

Algorithm 12.3: Transformer layer

Input: Set of tokens $\mathbf{X} \in \mathbb{R}^{N \times D} : \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$

Multi-head self-attention layer parameters

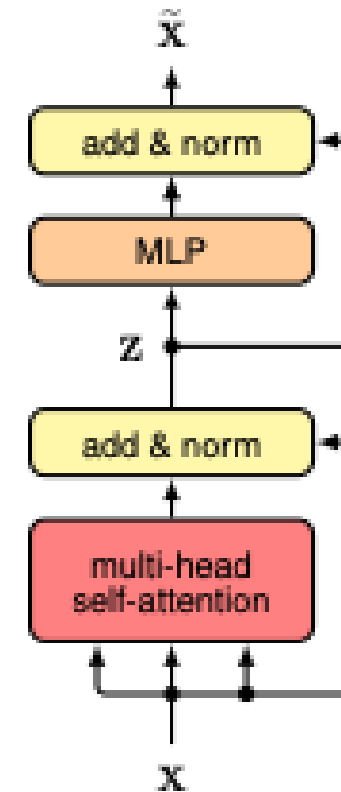
Feed-forward network parameters

Output: $\tilde{\mathbf{X}} \in \mathbb{R}^{N \times D} : \{\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N\}$

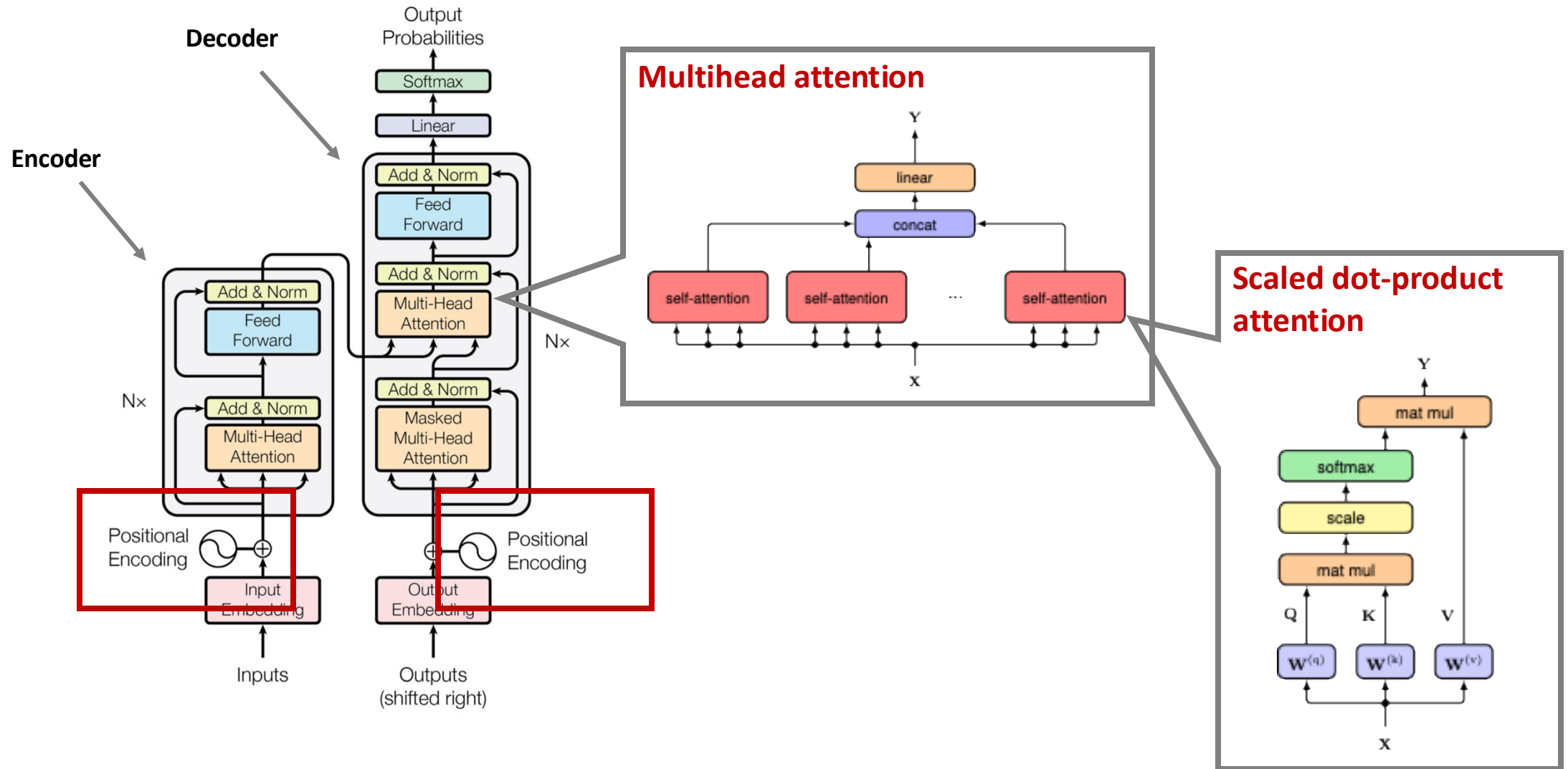
$\mathbf{Z} = \text{LayerNorm}[\mathbf{Y}(\mathbf{X}) + \mathbf{X}]$ // $\mathbf{Y}(\mathbf{X})$ from Algorithm 12.2

$\tilde{\mathbf{X}} = \text{LayerNorm}[\text{MLP}[\mathbf{Z}] + \mathbf{Z}]$ // shared neural network

return $\tilde{\mathbf{X}}$



Transformers overview



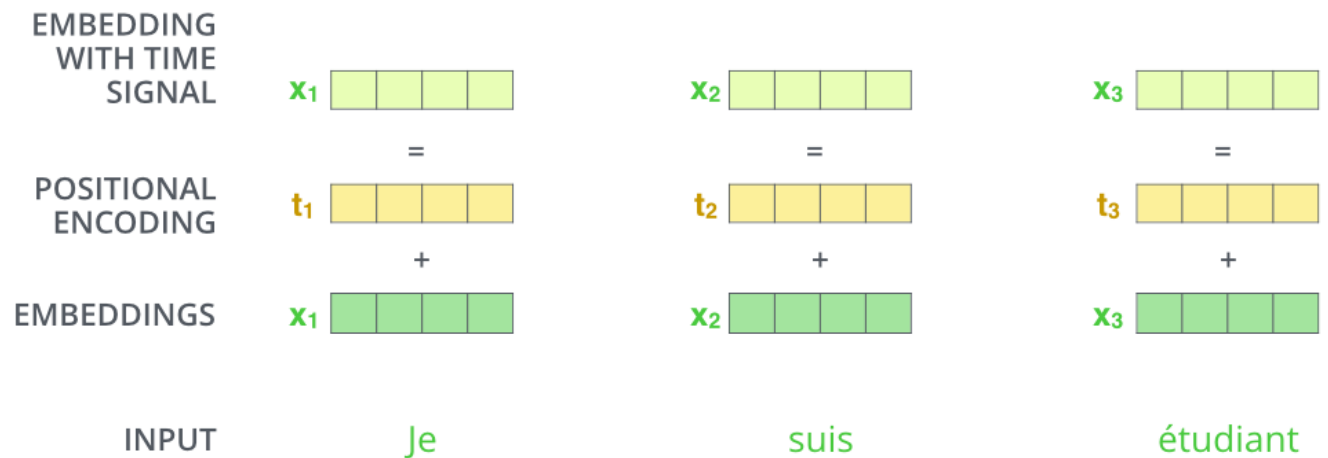
Positional encoding

Self-attention layer works on sets of vectors and it **doesn't know the order** of the vectors it is processing

The positional encoding has the **same dimension** as the input embedding

Adds a vector to each input embedding to give information about the **relative or absolute position** of the tokens in the sequence

These vectors follow a specific pattern



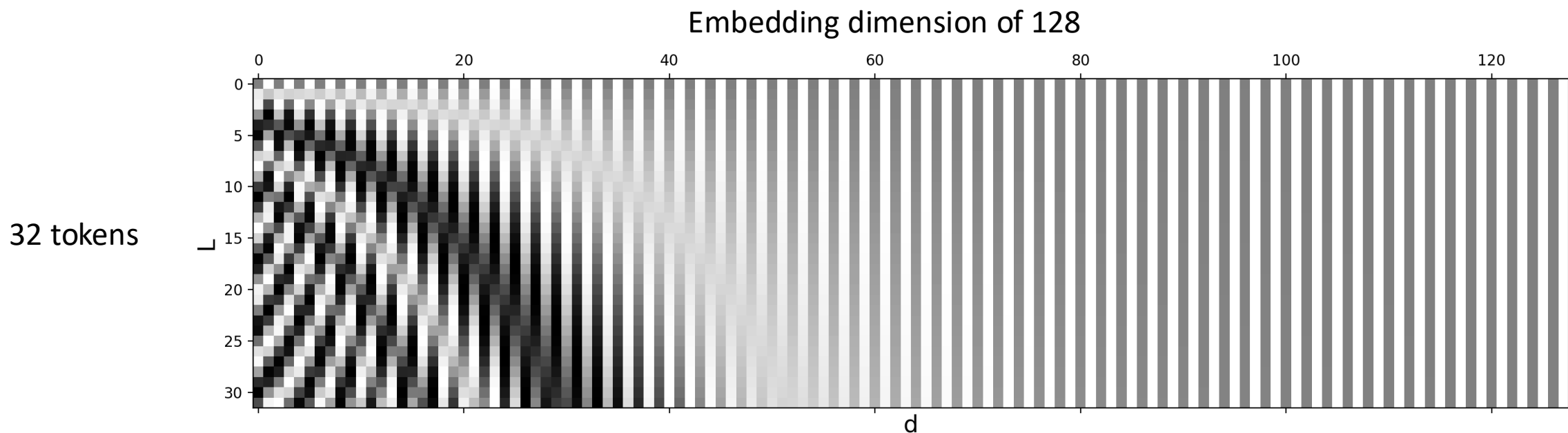
Positional encoding

What might this pattern look like?

Each row corresponds to a positional encoding of a vector.

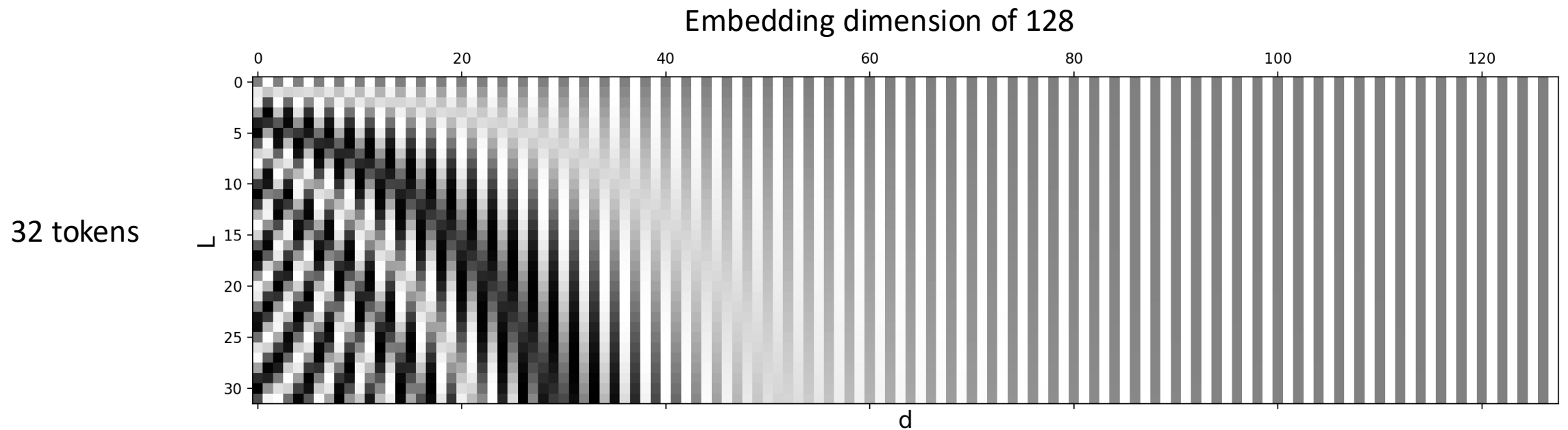
The first row would be the vector we'd add to the embedding of the first word in an input sequence.

Each position is uniquely encoded and the encoding can deal with sequences longer than any sequence seen in the training time.



Sinusoidal positional encoding with 32 tokens and embedding dimension of 128. The value is between -1 (black) and 1 (white) and the value 0 is in gray.

Positional encoding



Sinusoidal positional encoding - interweaves the two signals (sine for even indices and cosine for odd indices)

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

where pos is the position and i is the dimension,

Positional encoding – an example

Example:

Given the following *Sinusoidal positional encoding*, calculate the $PE(pos = 1)$ for the first five dimensions $[0, 1, 2, 3, 4]$. Assume $d_{model} = 512$

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

Solution:

Given $pos = 1$ and $d_{model} = 512$

At dimension 0, $2i = 0$ thus $i = 0$, therefore $PE_{(pos, 2i)} = PE_{(1, 0)} = \sin(1/10000^{0/512})$

At dimension 1, $2i + 1 = 1$ thus $i = 0$, therefore $PE_{(pos, 2i+1)} = PE_{(1, 1)} = \cos(1/10000^{0/512})$

At dimension 2, $2i = 2$ thus $i = 1$, therefore $PE_{(pos, 2i)} = PE_{(1, 2)} = \sin(1/10000^{2/512})$

At dimension 3, $2i + 1 = 3$ thus $i = 1$, therefore $PE_{(pos, 2i+1)} = PE_{(1, 3)} = \cos(1/10000^{2/512})$

At dimension 4, $2i = 4$ thus $i = 2$, therefore $PE_{(pos, 2i)} = PE_{(1, 4)} = \sin(1/10000^{4/512})$

Positional encoding - implementation

```
class PositionalEncoding(nn.Module):

    def __init__(self, d_model: int, dropout: float = 0.1, max_len: int = 5000):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model))
        pe = torch.zeros(max_len, 1, d_model)
        pe[:, 0, 0::2] = torch.sin(position * div_term)
        pe[:, 0, 1::2] = torch.cos(position * div_term)
        self.register_buffer('pe', pe)

    def forward(self, x: Tensor) -> Tensor:
        """
        Arguments:
            x: Tensor, shape ``[seq_len, batch_size, embedding_dim]``
        """
        x = x + self.pe[:x.size(0)]
        return self.dropout(x)
```

The constructor initializes the module and sets up the dropout layer with the given dropout probability.

`d_model` is the dimension of the embeddings (or the depth of the model).

`max_len` is the maximum expected length of the sequences.

Positional encoding - implementation

```
class PositionalEncoding(nn.Module):

    def __init__(self, d_model: int, dropout: float = 0.1, max_len: int = 5000):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model))
        pe = torch.zeros(max_len, 1, d_model)
        pe[:, 0, 0::2] = torch.sin(position * div_term)
        pe[:, 0, 1::2] = torch.cos(position * div_term)
        self.register_buffer('pe', pe)

    def forward(self, x: Tensor) -> Tensor:
        """
        Arguments:
            x: Tensor, shape ``[seq_len, batch_size, embedding_dim]``
        """
        x = x + self.pe[:x.size(0)]
        return self.dropout(x)
```

position is a tensor containing integers from 0 to max_len-1, representing each position in the sequence.

Positional encoding - implementation

```
class PositionalEncoding(nn.Module):

    def __init__(self, d_model: int, dropout: float = 0.1, max_len: int = 5000):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model))
        pe = torch.zeros(max_len, 1, d_model)
        pe[:, 0, 0::2] = torch.sin(position * div_term)
        pe[:, 0, 1::2] = torch.cos(position * div_term)
        self.register_buffer('pe', pe)

    def forward(self, x: Tensor) -> Tensor:
        """
        Arguments:
            x: Tensor, shape ``[seq_len, batch_size, embedding_dim]``
        """
        x = x + self.pe[:x.size(0)]
        return self.dropout(x)
```

div_term is a scaling term used to adjust the rate of the sinusoidal functions.

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

Positional encoding - implementation

```
class PositionalEncoding(nn.Module):

    def __init__(self, d_model: int, dropout: float = 0.1, max_len: int = 5000):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model))
        pe = torch.zeros(max_len, 1, d_model)
        pe[:, 0, 0::2] = torch.sin(position * div_term)
        pe[:, 0, 1::2] = torch.cos(position * div_term)
        self.register_buffer('pe', pe)

    def forward(self, x: Tensor) -> Tensor:
        """
        Arguments:
            x: Tensor, shape ``[seq_len, batch_size, embedding_dim]``
        """
        x = x + self.pe[:x.size(0)]
        return self.dropout(x)
```

The sinusoidal functions (sine for even indices and cosine for odd indices) are applied to the positions and the results are stored in `pe`. This creates a unique positional encoding for each position.

Positional encoding - implementation

```
class PositionalEncoding(nn.Module):

    def __init__(self, d_model: int, dropout: float = 0.1, max_len: int = 5000):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model))
        pe = torch.zeros(max_len, 1, d_model)
        pe[:, 0, 0::2] = torch.sin(position * div_term)
        pe[:, 0, 1::2] = torch.cos(position * div_term)
        self.register_buffer('pe', pe)

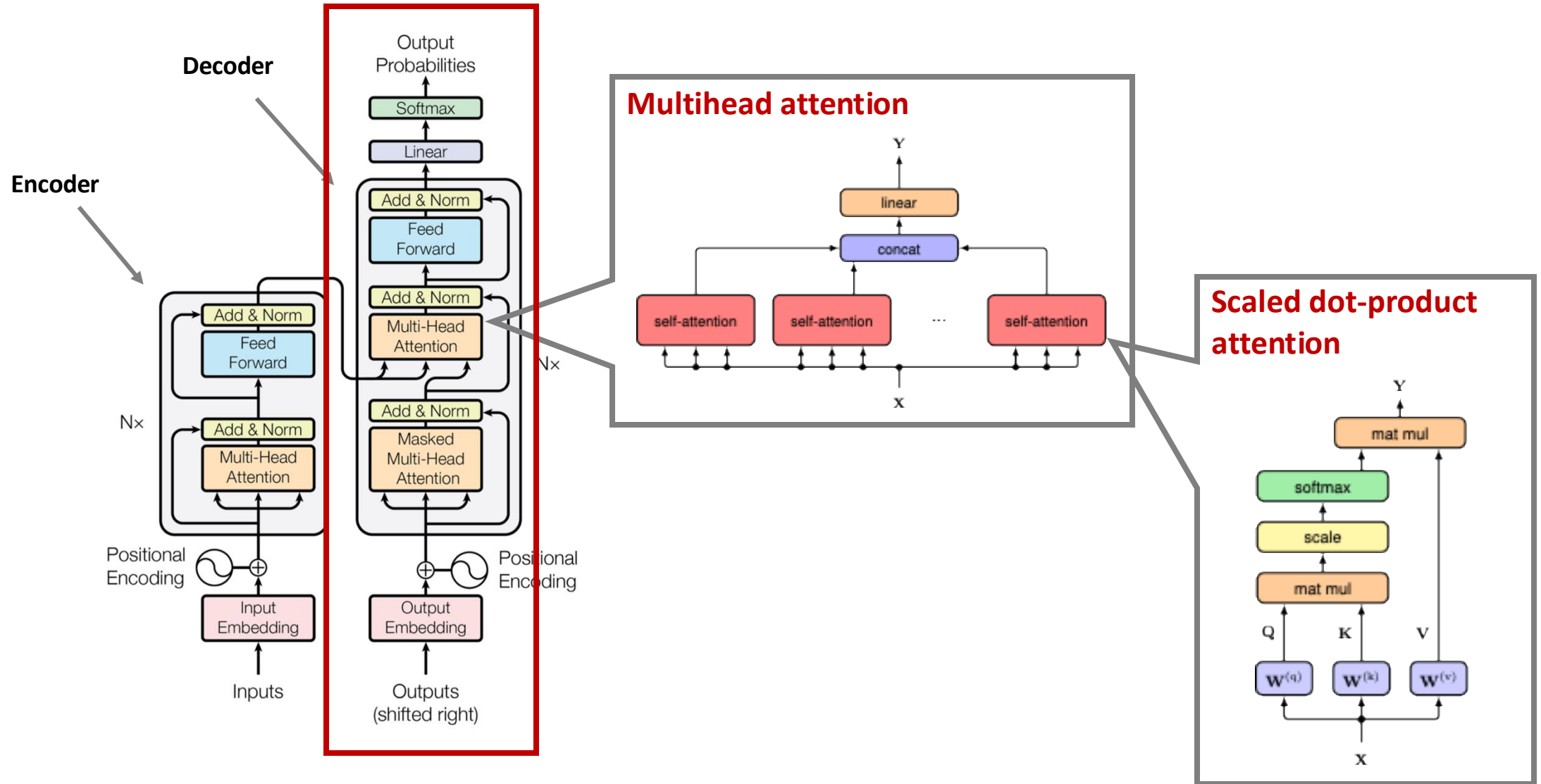
    def forward(self, x: Tensor) -> Tensor:
        """
        Arguments:
            x: Tensor, shape ``[seq_len, batch_size, embedding_dim]``
        """
        x = x + self.pe[:x.size(0)]
        return self.dropout(x)
```



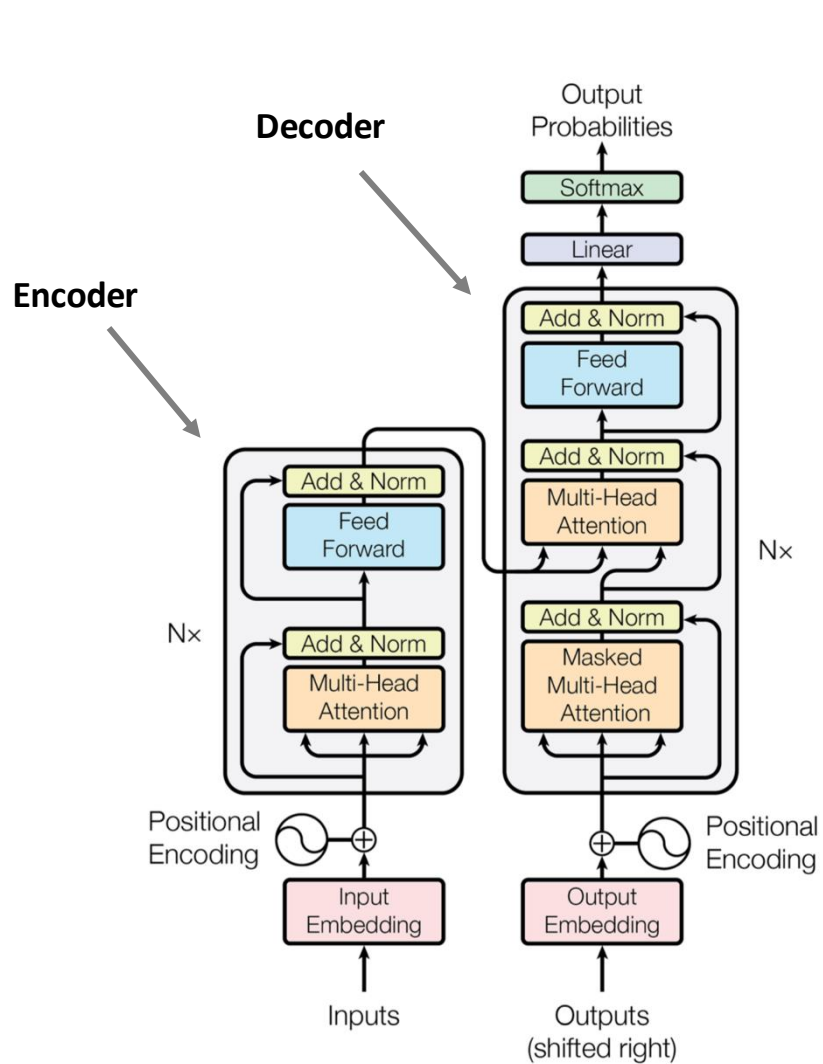
The positional encoding (self.pe) corresponding to the length of the input sequence is added to the input tensor x.

This addition operation effectively combines the positional information with the embeddings of the tokens.

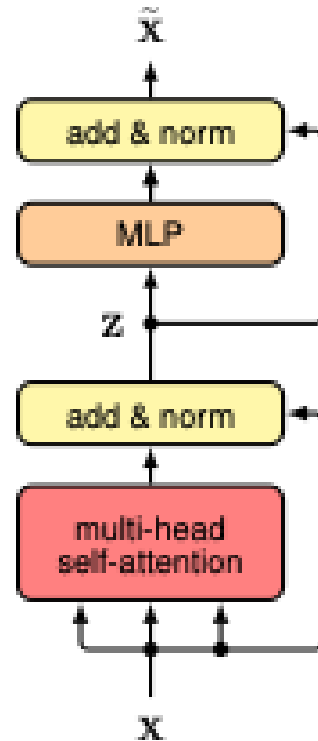
Transformers overview



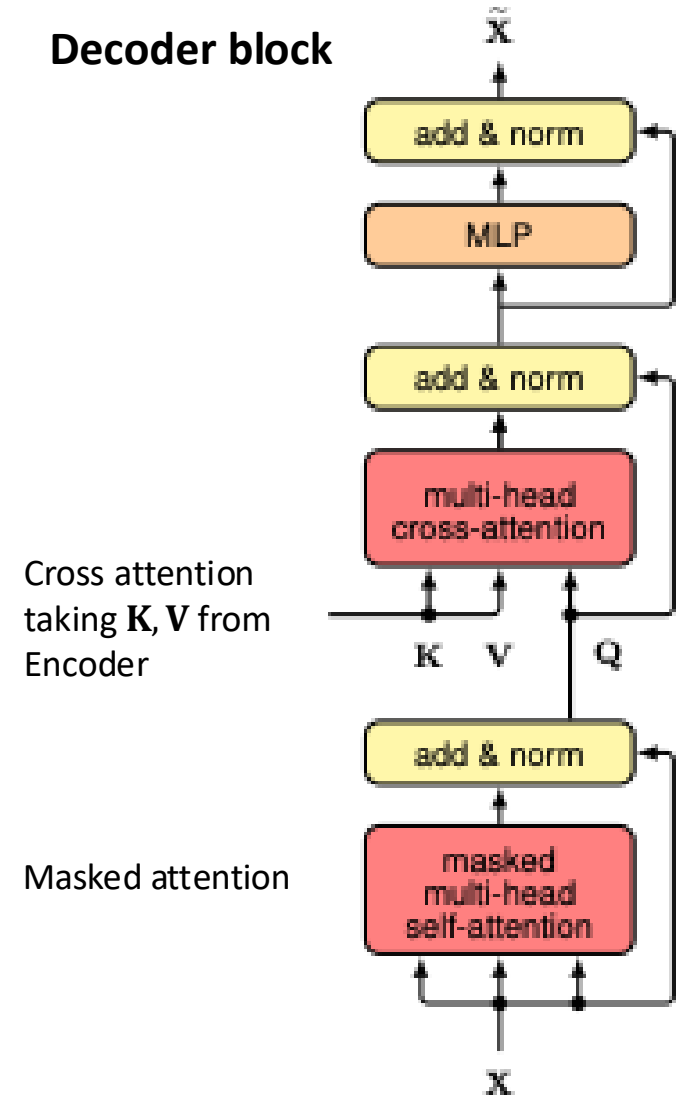
The decoder in sequence-to-sequence transformer



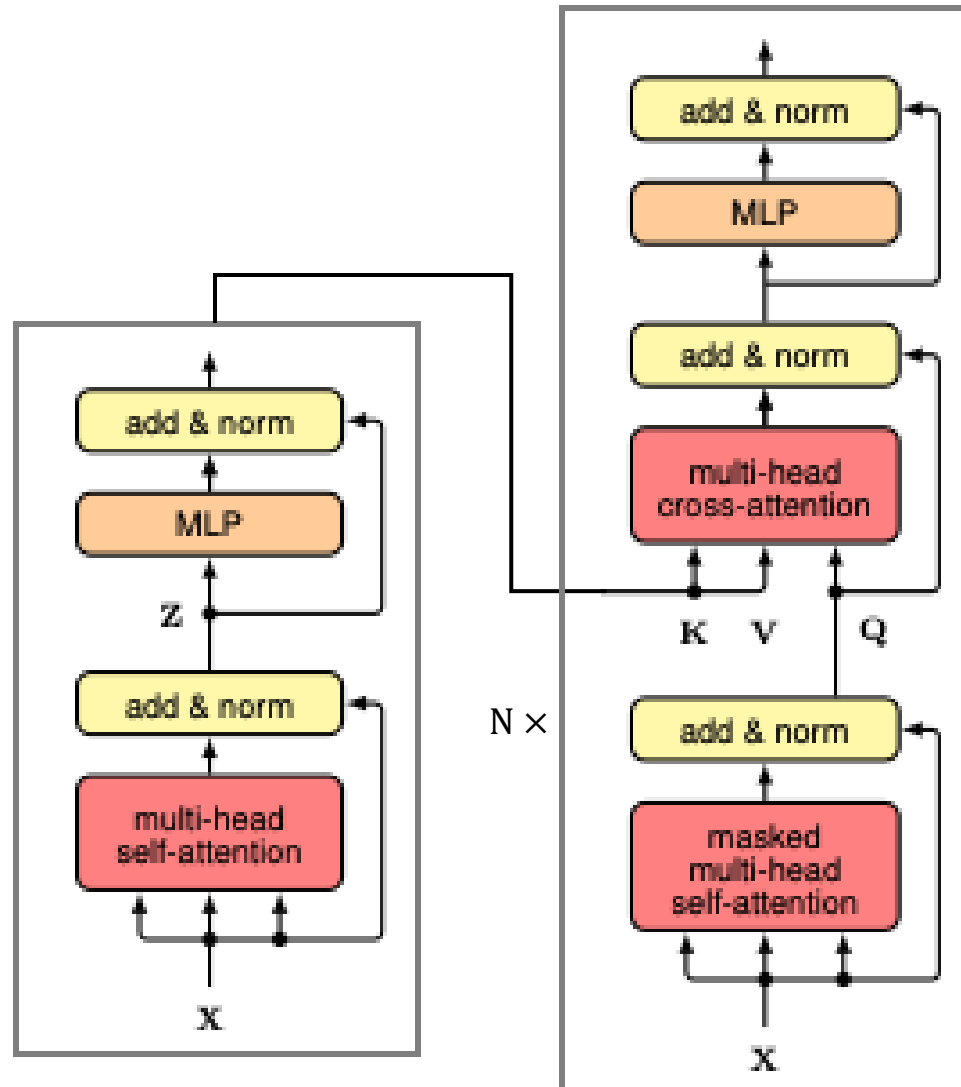
Encoder block



Decoder block



The decoder in sequence-to-sequence transformer



Cross attention

An encoder transformer is used to map the input token sequence into a suitable internal representation. Its output is broadcasted to all the decoder blocks in the decoder.

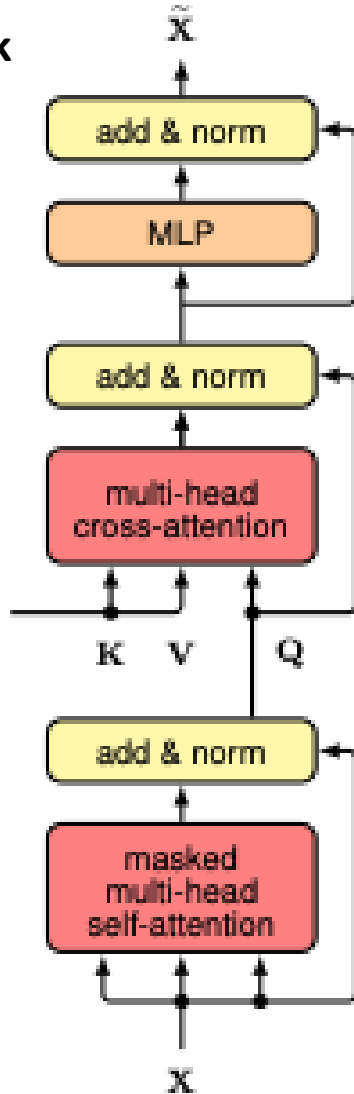
To incorporate the encoder output into the generative process for the output sequence, we use a modified form of the attention mechanism called *cross attention*.

To incorporate the encoder output into the generative process for the output sequence, we use a modified form of the attention mechanism called *cross attention*.

This is the same as self-attention except that although the query vectors come from the sequence being generated, the key and value vectors come from the sequence from encoder.

The decoder in sequence-to-sequence transformer

Decoder block



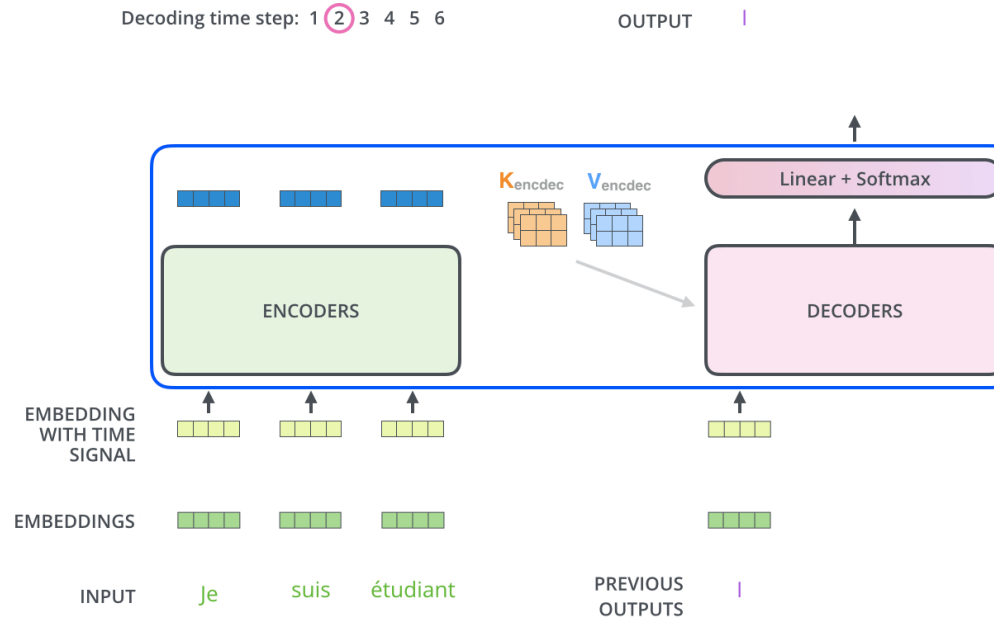
Cross attention
taking $\mathbf{K}, \mathbf{V}$ from
Encoder

Masked attention

Autoregressive model

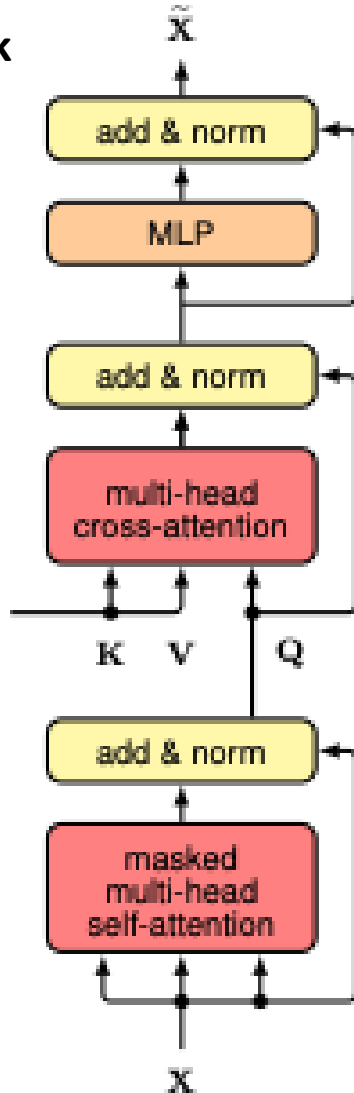
The model takes as input a sequence consisting of the first $n - 1$ tokens, and its corresponding output represents the conditional distribution for token n

$$p(\mathbf{x}_n | \mathbf{x}_1, \dots, \mathbf{x}_{n-1})$$



The decoder in sequence-to-sequence transformer

Decoder block



Cross attention
taking K, V from
Encoder

Masked attention

During the training we use **masked attention**

Naively we could process each such training sample independently using a forward pass through the model.

However, we can achieve much greater efficiency by processing an entire sequence at once so that each token acts both as a target value for the sequence of previous tokens and as an input value for subsequent tokens.

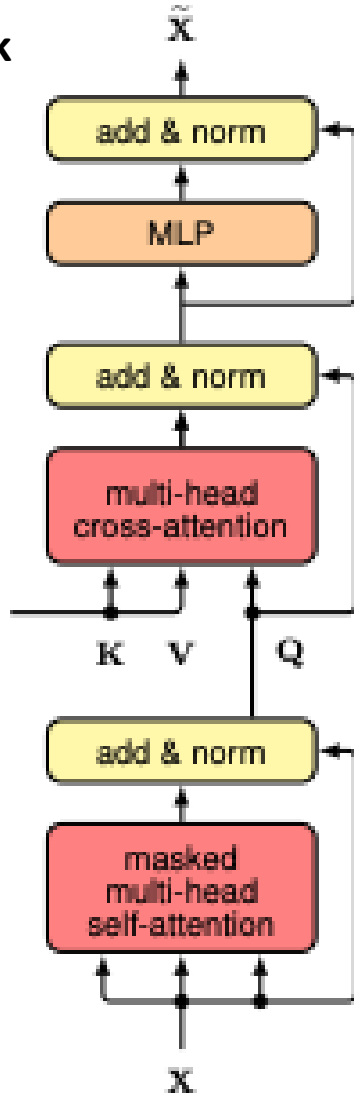
For example:

I swam across the river to get to the other bank.

Input	Target
I swam across	the
I swarm across the	river

The decoder in sequence-to-sequence transformer

Decoder block



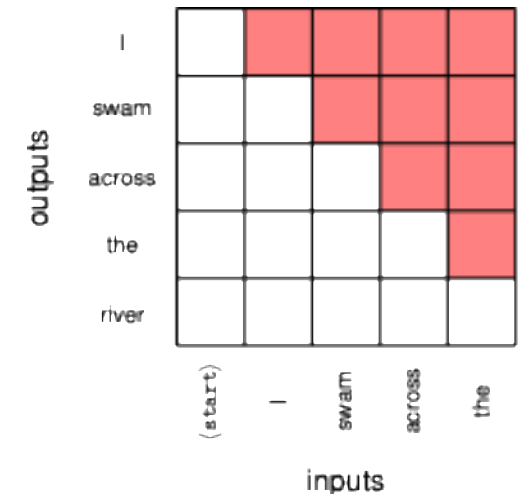
I swam across the river to get to the other bank.

Input	Target
I swam across	the
I swarm across the	river

To process these in **parallel** we have to ensure that the network **is not able to 'cheat' by looking ahead** in the sequence, otherwise it will simply learn to copy the next input directly to the output.

An illustration of the mask matrix for masked self-attention. Attention weights corresponding to the red elements are set to zero.

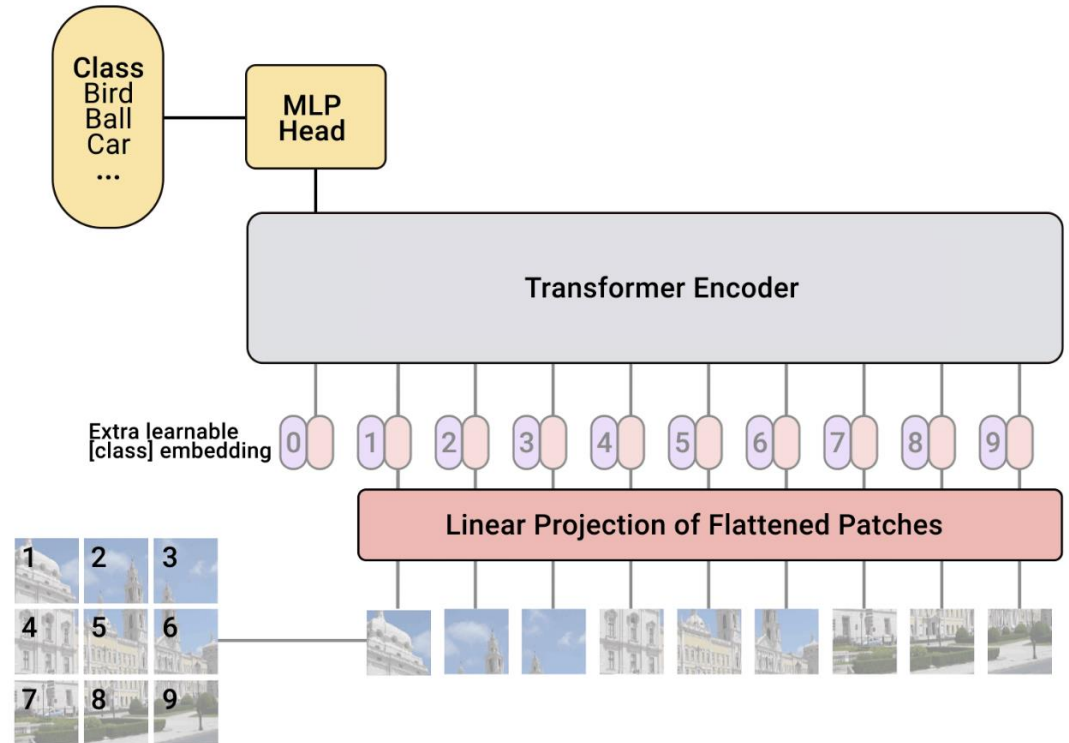
Thus, in predicting the token 'across', the output can depend only on the input tokens '(start)' 'I' and 'swam'.



Vision Transformer

Vision Transformer (ViT)

- Do not have decoder
- Reshape the image $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$ into a sequence of flattened 2D patches $\mathbf{x} \in \mathbb{R}^{N \times (P^2 \cdot C)}$
 - (H, W) is the resolution of the original image
 - C is the number of channels
 - (P, P) is the resolution of each image patch
 - $N = HW/P^2$ is the resulting number of patches



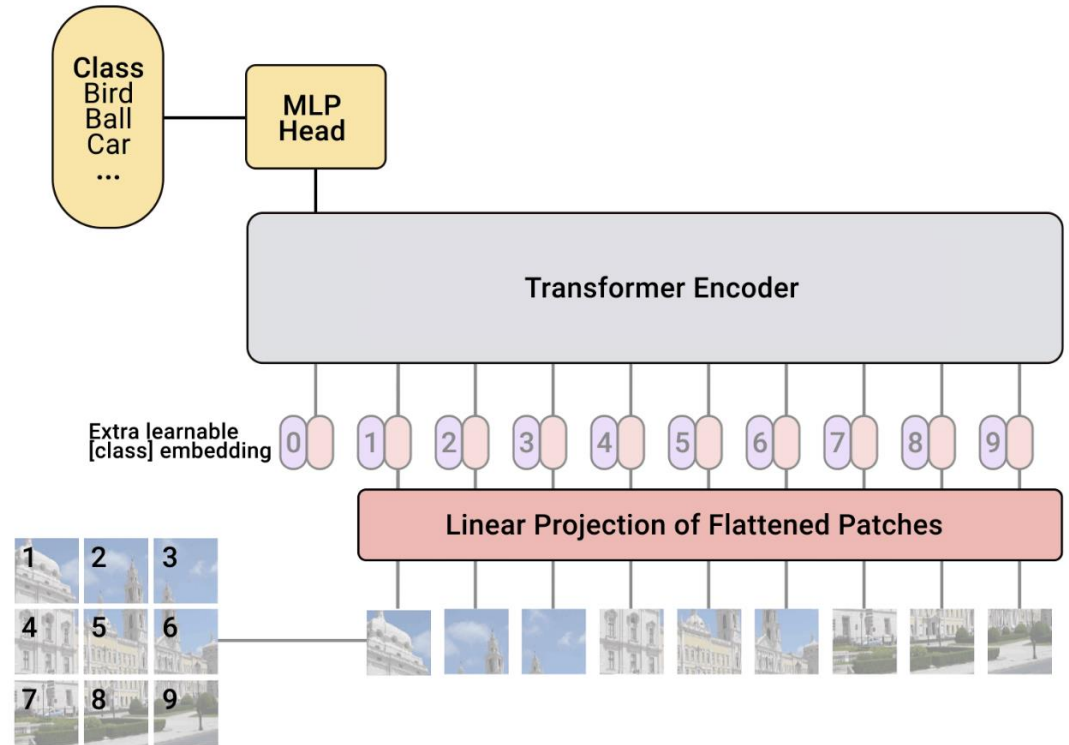
Vision Transformer (ViT)

Prepend a **learnable embedding** ($\mathbf{z}_0^0 = \mathbf{x}_{\text{class}}$) to the sequence of embedded patches

$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \cdots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{\text{pos}}$$

Patch embedding - Linearly embed each of them to D dimension with a trainable linear projection $\mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}$

Add **learnable position embeddings** $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$ to retain positional information



Vision Transformer (ViT)

Feed the resulting sequence of vectors $\mathbf{z}_0$ to a standard Transformer encoder

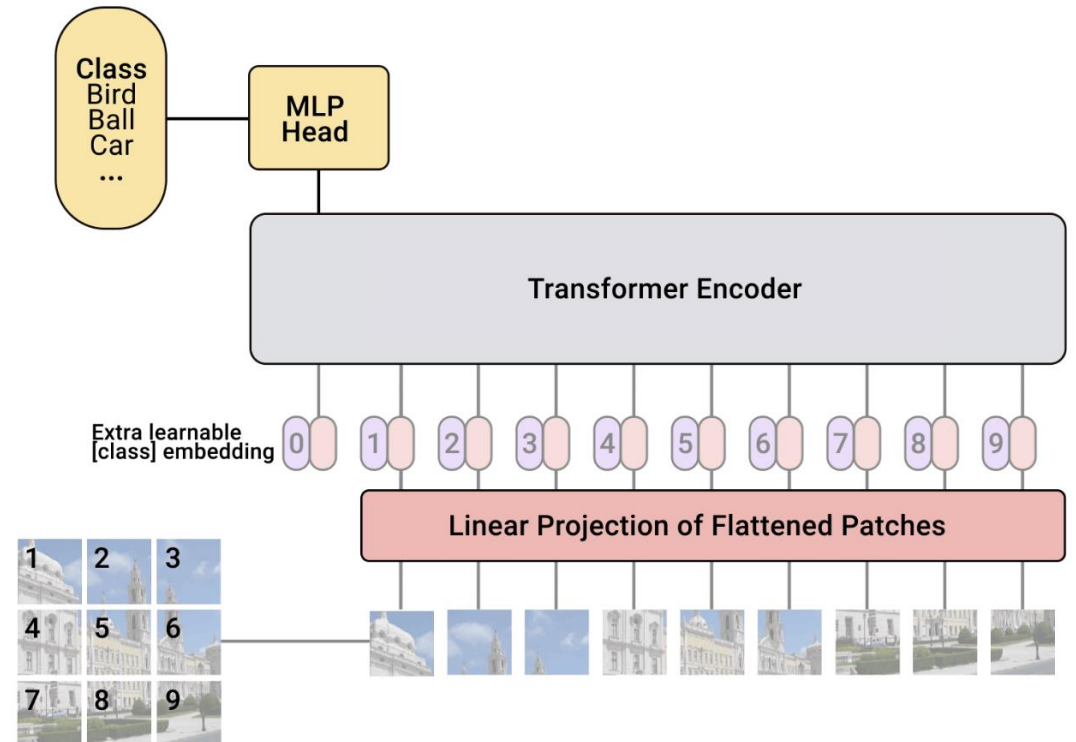
$$\begin{aligned} \text{Transformer Encoder} & \left\{ \begin{aligned} \mathbf{z}_0 &= [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{\text{pos}}, \\ \mathbf{z}'_\ell &= \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, \\ \mathbf{z}_\ell &= \text{MLP}(\text{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell, \end{aligned} \right. \\ \text{Classification Head} & \left\{ \begin{aligned} \mathbf{y} &= \text{LN}(\mathbf{z}_L^0) \end{aligned} \right. \end{aligned}$$

$$\mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}, \mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$$

$$\ell = 1 \dots L$$

$$\ell = 1 \dots L$$

The output of the additional [class] token is transformed into a class prediction via a small multi-layer perceptron (MLP) with tanh as non-linearity in the single hidden layer.



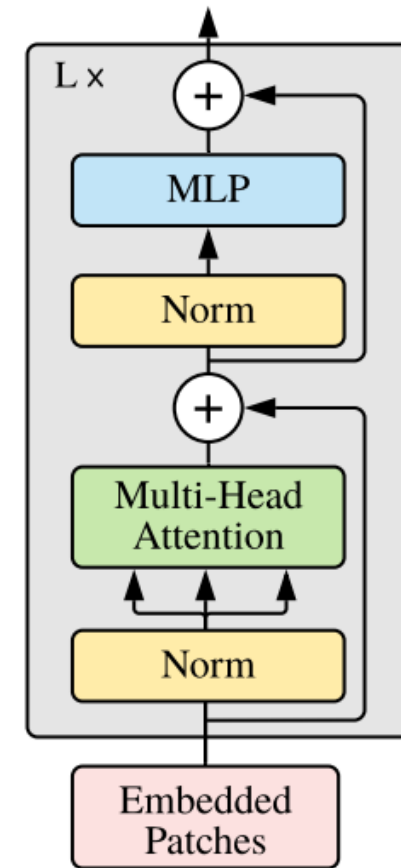
Vision Transformer (ViT)

Transformer Encoder

- Consists of a multi-head self-attention module (**MSA**), followed by a 2-layer MLP (with **GELU**)
- LayerNorm (LN) is applied before MSA module and MLP, and a residual connection is applied after each module.

$$\mathbf{z}'_{\ell} = \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1},$$

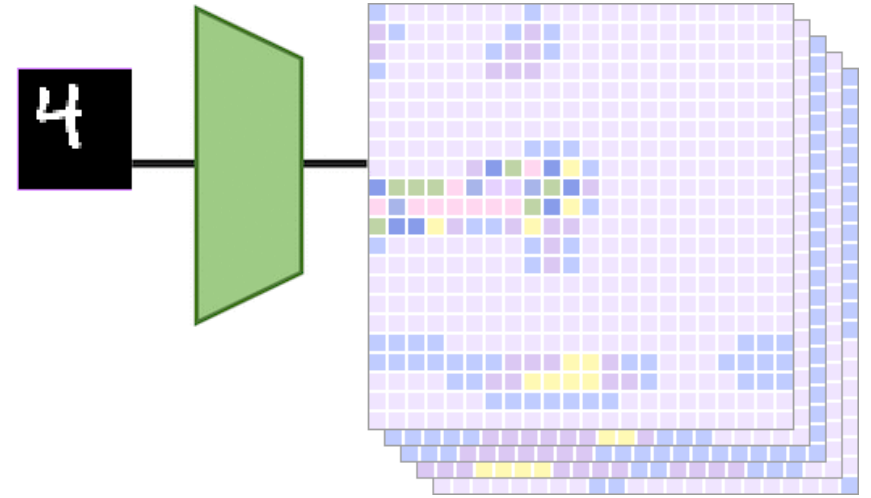
$$\mathbf{z}_{\ell} = \text{MLP}(\text{LN}(\mathbf{z}'_{\ell})) + \mathbf{z}'_{\ell},$$



Transformer Encoder

Inductive Bias

- ViT has much less image-specific inductive bias than CNNs
- Inductive bias in CNN
 - Locality
 - Two-dimensional neighborhood structure
 - Translation equivariance
- ViT
 - Only MLP layers are local and translationally equivariant. Self-attention layer is global
 - Two dimensional neighborhood is used sparingly – i) only at the beginning where we cut image into patches, ii) learnable position embedding (spatial relations have to be learned from scratch)



An equivariant mapping is a mapping which preserves the algebraic structure of a transformation.

A translation equivariant mapping is a mapping which, when the input is translated, leads to a translated mapping

Vision Transformer (ViT)

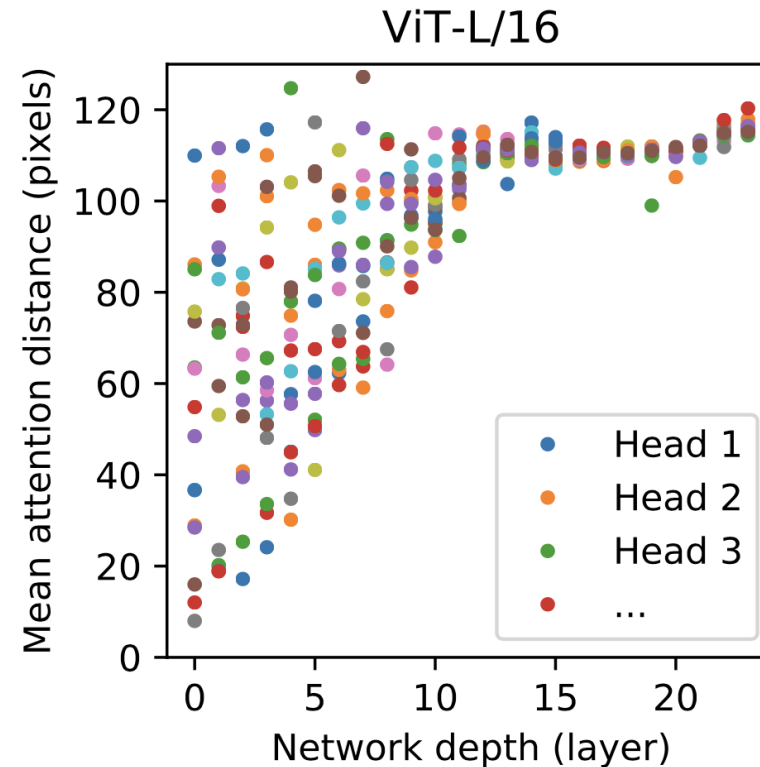
Examine the attention distance, analogous to receptive field in CNN

Compute the average distance in image space across which information is integrated, based on the attention weights.

Attention distance was computed for 128 example images by averaging the distance between the query pixel and all other pixels, weighted by the attention weight.

Each dot shows the mean attention distance across images for one of 16 heads at one layer. Image width is 224 pixels.

An example: if a pixel is 20 pixels away and the attention weight is 0.5 the distance is 10.

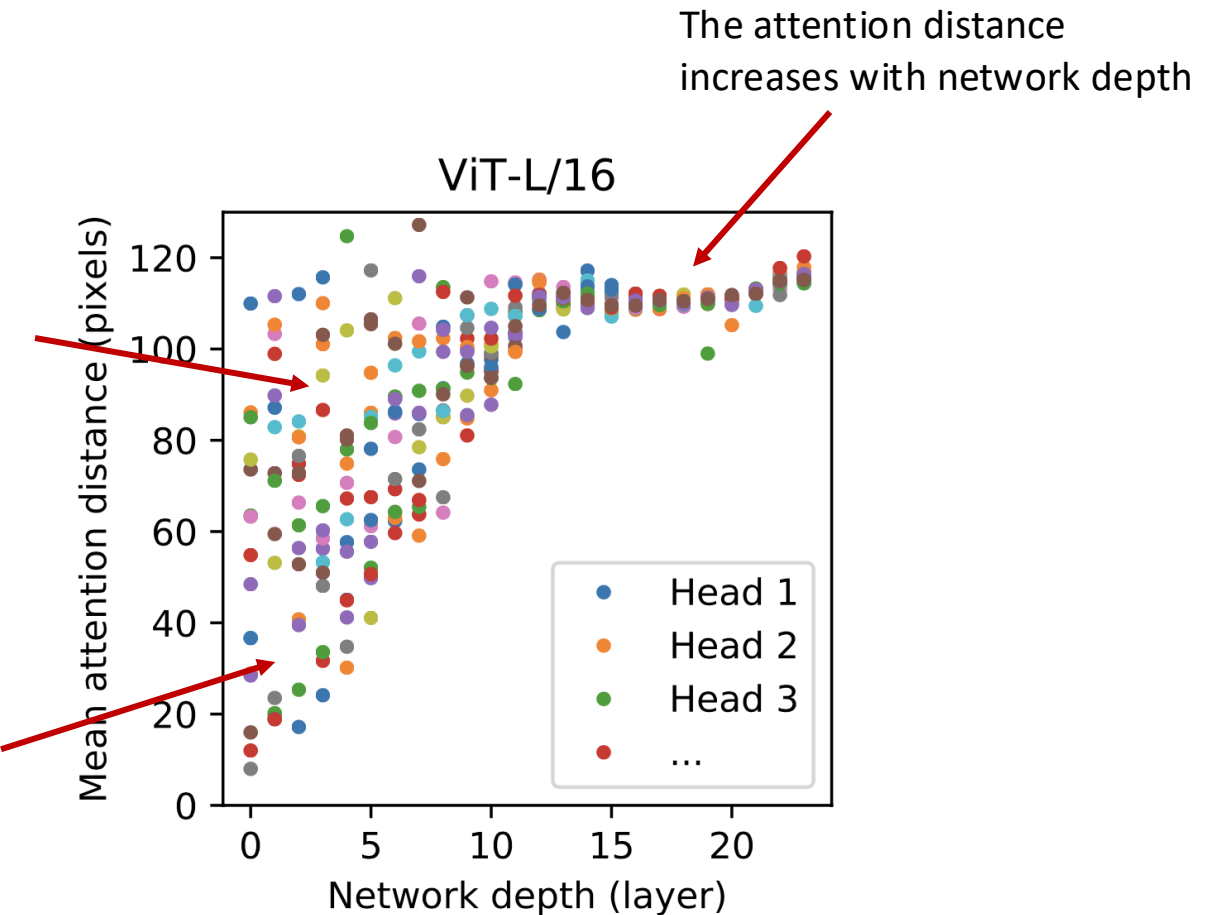


Vision Transformer (ViT)

Examine the attention distance, analogous to receptive field in CNN

Some heads attend to most of the image already in the lowest layers, showing the capability of ViT in integrating information globally

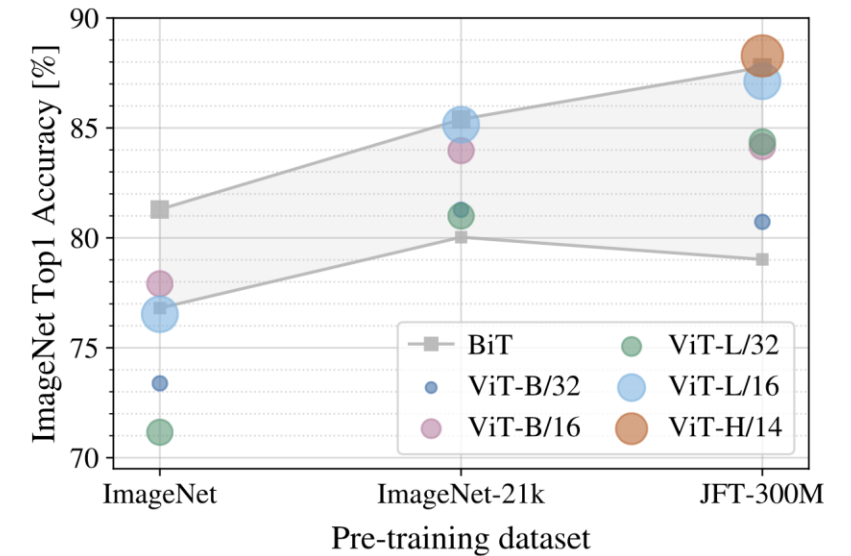
Other attention heads have consistently small attention distances in the low layers



Vision Transformer (ViT)

Performance of ViT

- ViT performs significantly worse than the CNN equivalent (BiT) when trained on ImageNet (1M images).
- However, on ImageNet-21k (14M images) performance is comparable, and on JFT (300M images), ViT outperforms BiT.
- ViT overfits the ImageNet task due to **its lack of inbuilt knowledge about images**



Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Vision Transformer (ViT)

ViT conducts **global self-attention**

- Relationships between a token and all other tokens are computed
- Quadratic complexity with respect to the number of tokens
- Not tractable for dense prediction or to represent a high-resolution image